GenMedX: Generative AI Healthcare Agent

Hamsalakshmi Ramachandran, Himani Shah, Kush Bindal, Shobhita Agrawal and Sugandha Chauhan

Department of Applied Data Science, San Jose State University

MSADI Project II Section 11 DATA 298B

Dr. Simon Shim

Fall 2025

4. Model Development

4.1 Model Proposals

4.1.1 Risk Prediction and Tests Recommendation

The Risk Prediction and Tests Suggestion element focuses on two main goals- forecasting emergency-department (ED) triage risk categories based on structured vital-sign information and unstructured chief complaints and creating suitable diagnostic test suggestions based on the patient's vitals and complaint. Every model enhances predictive capability, interpretability, and safety by integrating traditional machine-learning techniques with large-language-model (LLM) reasoning and retrieval-augmented generation (RAG).

Structured vital signs—systolic and diastolic blood pressure (SBP/DBP), oxygen saturation (O_2 Sat), respiratory rate (RR), heart rate (HR), temperature, and pain assessment—act as numerical indicators. Essential values that are absent are filled in with feature-specific medians, and clear missingness indicators (binary flags) are added to maintain the signal. Chief complaints are processed through token normalization (lowercasing, punctuation elimination, stopword removal) and transformed using a term-frequency × inverse-document-frequency (TF-IDF) vectorizer (≈ 500 features). The resulting sparse matrix combines both numerical vital signs and text-based complaint embeddings.

Inputs and outputs. Inputs consist of standardized vital-sign readings, primary complaint text, and optional notes from clinicians. Outputs consist of a forecasted triage-risk label along with a probability, a prioritized list of diagnostic tests, and concise contextual justifications appropriate for subsequent decision-support visualization.

Model 1 – Random Forest (Risk) + BioMistral-7B (Test Recommendation)

Targeted problem. Capturing nonlinear relationships between patient characteristics to enhance predictive accuracy beyond linear benchmarks. Creating adaptive, context-aware diagnostic test suggestions and assisting with clinical QnA activities.

Concept and features. Random forest consists of multiple decision trees trained on bootstrapped samples, which lowers variance and enhances generalization. It is ideal for diverse clinical data. BioMistral-7B is a large language model focused on the biomedical field, able to utilize both structured (vital signs) and unstructured (symptoms, questions) data to produce tailored results. In contrast to rigid guidelines, it adjusts nimbly to fresh or intricate situations.

Model architecture.. Ensemble classifier comprising 100 decision trees, with a maximum depth of 10 and balanced class weights to mitigate imbalance in risk labels. Autoregressive language model based on transformers, optimized with biomedical datasets, available via a GPU-supported inference endpoint. Outputs are subjected to canonicalization and fallback to enhance robustness.

Algorithm. For risk prediction, bootstrapped sampling → building a decision tree (utilizing Gini impurity) → combining tree outputs through majority voting → ultimate risk prediction with probability assessments. For tests, input patient characteristics → creation of a structured prompt → BioMistral suggests tests → standardization of test names → resort to regex/rules if necessary → produce structured JSON output.

Model 2 – XGBoost + Llama-3.1 Instruct

Targeted problem. Enhancing the accuracy of triage-risk prediction and the stability of models by utilizing gradient-boosted decision trees, while producing organized, instruction-compliant suggestions for diagnostic tests that ensure consistency and safety.

Concept and features. XGBoost (Extreme Gradient Boosting) utilizes sequential tree ensembles that progressively adjust the residual errors of earlier models, adeptly addressing nonlinear feature interactions and class imbalance typical in emergency-department information. It functions using standardized vital signs, fills in missing values, and employs TF-IDF representations of main complaints. Llama-3.1 Instruct, a transformer optimized for instruction, analyzes structured vital signs and textual context to provide concise, schema-adherent test panels. The combined hybrid approach merges quantitative assessment with natural-language reasoning for thorough triage assistance.

Model architecture. Gradient-boosted classifier made up of numerous shallow decision trees (maximum depth = 6, learning rate = 0.1) utilizing *scale_pos_weight* to counterbalance the under-representation of high-risk instances. The LLM layer is an autoregressive transformer specifically adapted for following instructions, functioning via a fixed JSON output format. The integrated system operates as a two-tier pipeline in which numerical probabilities and textual reasoning work in harmony.

Algorithm. Data preprocessing → scaling features and TF-IDF encoding → gradient boosting rounds that reduce binary log-loss → thresholding probabilities (≈ 0.46) for high-risk determination → creation of structured prompts based on patient context → Llama-3.1 Instruct produces diagnostic test suggestions → canonicalization and fallback protocols maintain consistent terminology and secure defaults → final JSON output combining risk label, probability, and suggested evaluations.

Model 3 – CatBoost + Qwen 2.5 Guarded

Targeted problem. Achieving a balance between predictive power and interpretability in triage-risk assessment while guaranteeing that diagnostic suggestions from the language model

are clinically safe and can be verified. The aim is to combine a calibrated probabilistic model with a restricted text generator that restricts unsupported or unclear outputs

Concept and features. CatBoost employs ordered boosting and target statistics encoding to manage categorical, missing, and continuous features inherently without one-hot encoding. This method reduces target leakage and ensures consistent performance on diverse ED data. Qwen 2.5 Guarded features a generation layer focused on safety: the model is fine-tuned on biomedical datasets and includes token-level filters and a JSON schema validator to prevent invalid, unnecessary, or fabricated medical terminology. The synthesis produces dependable numerical risk probabilities and textual results that conform to established clinical ontologies

Model architecture. The CatBoost classifier consists of 500 trees of depth 8, utilizing ordered boosting and Bayesian priors for categorical encodings. The model generates continuous risk scores adjusted via isotonic regression to improve the reliability of probabilities. Qwen 2.5 Guarded functions as a two-phase pipeline—initially a limited text generator adhering to a predetermined prompt format, followed by a guard module that ensures semantic and syntactic correctness. The inference interface transmits lightweight JSON objects.

Algorithm. Normalized input features → fill in medians → transform complaints with TF-IDF (≈ 500 dimensions). Train CatBoost with early stopping based on validation AUC; implement monotonic constraints for clinical interpretability (e.g., reduced oxygen saturation should not lower risk score). While inferring, calculated the calibrated probability $p \in [0, 1]$ and assign labels using a threshold of approximately 0.46. Created a protected prompt that includes a summary of risks, vital signs outliers, and context of the complaint. Qwen 2.5 Guarded creates candidate tests; the guard layer checks each token against an authorized terminology list.

Canonicalization combines synonyms and assigns urgency levels prior to outputting structured JSON.

Model 4 – CatBoost + Retrieval Memory (RAG Enhanced)

Targeted problem. Improving diagnostic test coverage and contextual memory by combining high-efficiency gradient boosting with retrieval-augmented reasoning that cites clinically comparable past cases.

Concept and features. CatBoost uses ordered boosting along with inherent handling of categorical features and strong regularization, providing consistent predictions on diverse ED data. Retrieval Memory enhances this by incorporating previous patient cases (vitals + primary concerns) into a compact vector repository. Upon the arrival of a new case, the system fetches the top-K closest records and compiles their test suggestions prior to concluding via the generative layer. This method combines evidence-backed recollection with model-informed flexibility.

Model architecture. A two-phase hybrid comprising (1) a CatBoost Classifier (depth = 8, iterations = 500, learning rate = 0.1) trained with a threshold of approximately 0.46 for risk differentiation, and (2) a generator enhanced by retrieval, constructed on vector embeddings (1000 × 307 matrix). The design preserves common preprocessing—imputation, scaling, TF-IDF text encoding—and incorporates a safety-rule layer to guarantee critical tests (CBC, BMP, Lactate, EKG) are always included.

Algorithm. Extraction of features and embedding → Risk scoring via CatBoost → Vectorization of query embeddings → Cosine similarity search in memory storage → Identification of top-3 comparable cases → Aggregation of suggested tests weighted by urgency and frequency → Implementation of rule-based safety modifications → Creation of a

consolidated testing plan with priority labels (STAT, Urgent, Routine) → Final JSON output with risk probability, classification, and a list of recommendations.

Integration of Risk and Tests

To enhance usability, the risk prediction and test recommendation components were merged into one decision-support agent. The agent produces the anticipated risk label, risk probability, reasoning (main complaint and essential vital signs), suggested diagnostic tests, and clinical warning signs. This integration guarantees that risk scores and test recommendations are not isolated but are instead contextualized together, in line with how clinicians make swift triage decisions.

4.1.2 Patient's ED Visits Summarizer

This module generates accurate, empathetic, and discharge-ready summaries of a patient's Emergency Department (ED) visit. It transforms structured hospital data into a concise, patient-friendly narrative that both clinicians and patients can easily interpret.

Concept and Features. The Patient's ED Visit Summarizer aims to convert structured ED visit data into clear and concise summaries that describe the patient's condition, key findings, and outcome. The goal is to transform a compact serialization of ED visit fields into a natural, readable narrative that a patient can easily understand. The system integrates a fine-tuned T5-small Transformer model with a deterministic rule-based fallback component to ensure reliability and factual accuracy. The T5-small model, comprising approximately 60 million parameters, was chosen for its lightweight architecture and computational efficiency—small enough for CPU inference while maintaining strong summarization quality and low-latency response. The summarizer follows an ML-first-then-validate workflow, where the T5-small model generates the summary, a validator checks for factual and structural accuracy, and a

deterministic summary builder provides a fallback when validation fails. This hybrid design balances linguistic fluency, factual correctness, and system reliability under real-world clinical constraints.

Model Architecture. The summarizer employs a fine-tuned T5-small encoder–decoder Transformer as its core component for generating patient-readable narratives from structured ED data. The model receives serialized information such as arrival details, chief complaint, vitals, disposition, and diagnoses as input text and outputs concise, coherent summaries describing the visit. The architecture consists of three integrated modules: (1) the T5-small summarizer, a text-to-text model that generates the initial narrative from structured fields; (2) the validator module, which performs post-processing checks to ensure factual completeness, field coverage, and proper formatting of numerical units (°F, /min, %, mmHg); and (3) the deterministic summary builder, a rule-based fallback mechanism that constructs factual summaries using predefined templates whenever the ML output fails validation or encounters an error. The trained model artifacts, including configuration files, tokenizer, and model weights, are stored under the /model directory for deployment. The validator also enforces correct syntax and detects contradictions such as stating both “admitted” and “discharged home.” If validation fails or a timeout occurs, the fallback system automatically returns a safe, structured summary while logging the issue for further analysis.

Algorithm and Approach. The summarizer model operates through a structured multi-stage workflow that combines machine learning-based text generation with deterministic validation. The process begins with data serialization, where structured ED visit information, such as patient demographics, arrival mode, vitals, medications, and diagnoses is transformed

into a standardized text format. Each stay is represented as a compact, ordered string that includes explicit units and consistent field separators, for example:

date={YYYY-MM-DD}; arrival={AMBULANCE|WALK IN}; complaint={text}; vitals[T={temp}F HR={hr}/min RR={rr}/min SpO2={sat}% BP={sbp}/{dbp}mmHg]; disposition={HOME|ADMITTED}; primary_dx={icd_title_primary}; other_dx=[title1; title2; ...]. Missing fields are omitted without placeholders, and trivial normalizations such as consistent spacing and letter casing are applied.

The serialized input string serves as the context for the T5-small encoder–decoder Transformer, which generates a one-to-two-sentence abstractive summary describing the patient's ED visit in clear and empathetic language. Once the summary is generated, it proceeds through the validation phase, where the validator module ensures factual completeness, verifies field coverage, checks unit consistency, and enforces syntax correctness. If the output fails validation or an inference error occurs, the system invokes the deterministic summary builder, which produces a fallback summary using predefined templates and explicit field values. This workflow ensures that every input produces a valid, factual output—either machine-generated or template-based maintaining fluency and precision even under data or resource constraints.

Training Data and Optimization. The training dataset was derived from the project CSV and transformed into Instruction–Context–Response (ICR) triplets to guide supervised learning. The Instruction was "Summarize this ED visit for the patient in one to two sentences." The Context contained the serialized ED data string, and the Response represented the target summary. The model was fine-tuned from the T5-small base checkpoint using the AdamW optimizer with an initial learning rate of approximately 5e-5, linear decay scheduling, and a weight decay of 0.01. Training used the cross-entropy loss function with teacher forcing for 15

epochs, and decoding employed greedy search with a maximum of about 60 generated tokens to ensure concise, well-structured summaries. The tokenizer preserved field separators for accurate contextual understanding. The best-performing checkpoint was selected based on validation loss and ROUGE metrics, achieving a validation loss of 0.1827 and ROUGE-1/ROUGE-2/ROUGE-L scores of 93.66, 90.75, and 93.64, respectively.

Expected Outcome. The expected outcome of the Patient's ED Visit Summarizer is to produce concise, accurate, and patient-friendly summaries that transform complex structured medical data into easy-to-read natural-language narratives. The summaries explain the patient's reason for visit, clinical findings, treatment provided, and outcome in clear terms while maintaining factual integrity and clinical correctness. By combining the generative fluency of the T5-small model with the reliability of the rule-based validator and fallback mechanisms, the system ensures that every summary is both linguistically coherent and factually valid. This approach minimizes the risk of incomplete or inconsistent information, improves patient comprehension and communication, and supports healthcare providers by automating documentation workflows. Ultimately, the model provides a scalable, safe, and consistent summarization solution optimized for real-world clinical use.

4.1.3 Disease Classifier

Concept and Features

This module is designed to accurately predict human diseases based on a user's input symptoms by leveraging ensemble machine learning techniques. The goal of this system is to assist early disease detection and provide preliminary diagnostic support by analyzing multiple symptom combinations. The model is trained on a structured medical symptoms dataset sourced

from Hugging Face, which includes 132 symptom features mapped to various disease prognoses. Key features of this module include:

- **Symptom-based disease prediction:** Users input symptoms in natural language (comma-separated), and the system identifies the most probable disease.
- **Multi-model ensemble learning:** The classifier integrates Support Vector Machine (SVM), Gaussian Naive Bayes, and Random Forest models to enhance prediction reliability.
- **Automated preprocessing and label encoding:** The model pipeline handles missing values and transforms categorical disease names into numerical labels using LabelEncoder, ensuring compatibility with ML algorithms.
- **Balanced dataset handling:** The underlying dataset is balanced, with 120 samples per disease, ensuring unbiased model training and evaluation.
- **Voting-based ensemble prediction:** A VotingClassifier combines the predictions of individual models using majority voting, improving robustness and minimizing single-model bias.

Model Architecture

The Disease Classifier employs a hybrid *ensemble-based architecture* that fuses multiple supervised learning algorithms to ensure high accuracy and generalization. The architecture is composed of three core components:

1. **Base Learners:**
    - **Support Vector Classifier (SVC):** Captures complex nonlinear relationships among symptoms.

- **Gaussian Naive Bayes (GNB):** Provides fast, probabilistic predictions assuming feature independence.
- **Random Forest Classifier (RFC):** Aggregates multiple decision trees to reduce overfitting and variance.

2. **Voting Mechanism:**

   The predictions from all three classifiers are combined using a hard voting strategy, where the final disease label is determined by the majority vote. This reduces error propagation from individual models and ensures consistent outputs even when one model misclassifies a sample.

3. **Symptom Encoder Layer:**

   The model uses a symptom index dictionary to convert textual symptoms (e.g., "Skin Rash", "Itching") into a 132-dimensional binary feature vector, which is then passed to the ensemble model for classification.

This modular design enables flexibility — individual models can be retrained or replaced without disrupting the entire system, making it scalable for integration with larger medical diagnostic frameworks.

Algorithm and Approach

The Disease Classifier follows a structured pipeline to ensure accuracy, interpretability, and robustness:

1. **Data Preprocessing:**
   - The dataset is read and cleaned by removing null columns.

- Disease labels (prognosis) are encoded into numerical values using LabelEncoder.
- The dataset is split into 80% training and 20% testing sets.

2. **Model Training and Cross-Validation:**
   - Each of the three classifiers (SVM, Naive Bayes, and Random Forest) is trained individually.
   - 10-fold cross-validation is performed to ensure model reliability and prevent overfitting.
   - All models achieved a mean accuracy score of 90% on the cross-validation sets, indicating strong performance on balanced data.

3. **Ensemble Prediction Logic:**
   - Predictions from individual classifiers are aggregated using the mode of predictions, ensuring robustness against minor deviations among classifiers.
   - Later, a VotingClassifier was implemented to formalize the ensemble learning process, combining the base estimators into a unified model for efficient deployment.

3. **Deployment and Inference:**
   - The trained voting classifier is serialized using **Joblib** for future use.
   - During inference, user-provided symptoms are preprocessed into binary input vectors.
   - The voting classifier predicts the most probable disease by leveraging the collective decision of all three base models.

This multi-stage process combines statistical learning and ensemble logic, ensuring high predictive reliability even for overlapping symptom patterns.

Expected Outcome

The expected outcome of the Disease Classifier is to produce accurate, interpretable, and consistent disease predictions based on symptom inputs. The model consistently achieved 100% accuracy across training, validation, and testing datasets, demonstrating its robustness on the balanced dataset.

The system outputs the most probable disease name along with individual model predictions, allowing transparency in diagnostic inference. By integrating ensemble learning, it minimizes the risk of single-model overfitting and ensures stable results across diverse symptom combinations.

In practical scenarios, this module can serve as a decision-support tool for telemedicine platforms, healthcare assistants, or medical chatbot systems — helping non-specialists and patients to identify potential diseases early and seek medical consultation accordingly. Its modular design, scalability, and explainable outputs make it suitable for real-world deployment in digital health ecosystems.

4.1.4 Medical Q&A

The Medical Question Answering (MedQA) Agent functions as a specialized reasoning component within the broader GenMedXAgent architecture, focusing on clinical interpretation, contextual retrieval, and structured response generation for open-ended medical questions. Its design integrates several approaches: large language model (LLM) prompting, multi-agent

simulation, and rule-based logic to balance adaptability, reliability, and speed. Each model variation within the MedQA agent addresses a different layer of complexity in medical question answering, providing a robust and explainable decision-support system.

Model 1 – Single-LLM Expert with Prompting Techniques

Targeted problem. Generating fluent and clinically accurate medical responses with controllable reasoning depth.

Concept and features. A locally or API-hosted transformer LLM is prompted through multiple strategies, zero-shot, few-shot, chain-of-thought (CoT), self-consistency, and CoT combined with self-consistency to study how prompt design affects reasoning transparency, factual accuracy, and computational efficiency.

Model architecture. An autoregressive text-generation pipeline equipped with modular prompt templates and schema-validated JSON output.

Algorithm. Input question → construct prompt using chosen technique → generate response (with or without sampling) → if self-consistency enabled, aggregate multiple reasoning paths by majority vote → output structured answer containing reasoning trace and safety tag.

Inputs and outputs. Inputs consist of natural-language medical questions in free-text or multiple-choice format; outputs are structured JSON responses that include the final answer, a concise reasoning explanation, and confidence or safety notes.

Model 2 – Multi-Agent Simulation

Targeted problem. Addressing complex or ambiguous questions that require multiple expert perspectives and conservative consensus.

Concept and features. The system emulates multidisciplinary clinical teams - such as Internal Medicine, Emergency Care, and Family Practice - by assigning each role a dedicated

prompt. Parallel LLM instances generate independent opinions that are synthesized by a coordinator agent; a final decision agent consolidates these into one recommendation once consensus or stopping criteria are reached.

Model architecture. Multiple parallel LLM instances conditioned by specialist prompts and governed by orchestration logic that manages discussion rounds, reconciliation, and synthesis.

Algorithm. Distribute query to role-specific prompts → collect independent reasoning responses → meta-agent synthesizes outputs → decision agent reconciles discrepancies → produce unified, schema-formatted recommendation.

Inputs and outputs. Input is the same clinical question distributed across roles; outputs include each specialist's reasoning summary and the final consensus answer.

Model 3 – Rule-Based Baseline

Targeted problem. Delivering instantaneous, deterministic responses for definitional or pattern-matched questions when full reasoning is unnecessary.

Concept and features. A lightweight dictionary-driven mapping system links medical terms to canonical definitions or procedural guidance, providing consistent, latency-free answers suitable for triage or low-risk use cases.

Model architecture. Simple Python script utilizing regex and lookup tables for clinical terminology normalization.

Algorithm. Preprocess question text → match keywords against medical dictionary → retrieve predefined response → append standardized safety disclaimer → format as validated JSON.

Inputs and outputs. Inputs are text-based medical questions; outputs are pre-verified factual statements or fallback advisories.

Model 4 – Retrieval-Augmented LLM (RAG) (Proposed)

Targeted problem. Reducing hallucinations, grounding answers in citable evidence, and maintaining relevance to up-to-date clinical guidance.

Concept and features. The LLM is conditioned on text passages retrieved from curated biomedical corpora such as PubMed abstracts, clinical guidelines, and institutional notes indexed in BigQuery and Vertex AI Vector Search. The agent employs dual retrieval lexical BM25 and dense embedding search with optional re-ranking, and constructs cite-aware prompts that restrict responses to retrieved evidence.

Model architecture. A two-stage pipeline combining retrieval and generation. Retrieved context segments feed directly into the transformer generator, which appends citation tokens. A confidence gate suppresses or reroutes answers when retrieval coverage is insufficient.

Algorithm. Input question → retrieve top-k relevant passages → re-rank by similarity → compose cite-aware prompt → generate answer limited to retrieved text → return answer with citations, rationale, and retrieval coverage score; if coverage < threshold, forward query to the Multi-Agent Simulation for safe handling.

Inputs and outputs. Inputs are free-text medical questions; outputs are grounded answers with concise rationales, citation identifiers, retrieval coverage, and model confidence scores.

Together, these four configurations establish a layered framework in which the Single-LLM Expert ensures adaptability and detailed reasoning, the Multi-Agent Simulation delivers reliability through consensus, the Rule-Based Baseline guarantees speed and safety, and

the Retrieval-Augmented LLM provides factual grounding supported by explicit evidence. This multi-method design enables the *GenMedX* MedQA module to respond effectively across the spectrum of clinical question complexity while preserving transparency and medical accuracy.

4.2 Model Supports

4.2.1 Risk Prediction and Tests Recommendation

Platform and environment. Development was primarily done in Python 3.11 utilizing Google Colab Pro+ (A100 GPU runtime) and a local Windows 10 machine with 32 GB RAM and a 12-core CPU. All experiments were conducted in regulated virtual settings to guarantee reproducibility. Colab provided GPU acceleration for inference with large language models, while local execution enabled preprocessing, evaluation, and packaging of artifacts. Risk-prediction trials were conducted utilizing Scikit-learn 1.6, XGBoost 2.0, and CatBoost 1.2.5, while matrix computations were enhanced by NumPy 1.26 and SciPy 1.11.

Generative models, such as BioMistral-7B, Llama-3.1 Instruct, and Qwen 2.5 Guarded, were utilized via the Hugging Face Transformers interface with mixed-precision inference leveraging bitsandbytes (4-bit and 8-bit quantization) to lower VRAM usage. Deployment and serving utilized FastAPI 0.111, encapsulated within container images based on Python slim foundations and managed via Vertex AI Model Registry.

Tools. Model creation and testing were carried out utilizing a strong open-source Python framework. Scikit-learn was utilized for data preprocessing, TF-IDF feature extraction, evaluating models, and baseline classifiers like Random Forest. XGBoost and CatBoost offered sophisticated gradient-boosting solutions for Models 2-4, enabling capabilities for managing imbalanced data and analyzing feature importance. Numerical calculations and sparse-matrix tasks were executed using NumPy and SciPy, whereas Pandas aided in data manipulation,

validation, and feature development. The Hugging Face Transformers library offered a cohesive interface for BioMistral-7B, Llama-3.1 Instruct, and Qwen 2.5 Guarded in large-language-model integration, with quantized inference supported by bitsandbytes in 4-bit and 8-bit formats to decrease GPU memory consumption. Matplotlib and Seaborn were utilized for visualization and diagnostic assessment, aiding in the creation of confusion matrices, ROC/PR curves, and comparative bar charts for model metrics. Persistence and reusability of models were accomplished via joblib, saving serialized preprocessing elements (*imputer.pkl, scaler.pkl, tfidf_fresh.pkl*) and trained models (*xgb.pkl, catboost.cbm, etc.*). Parquet and pickle formats were utilized to save intermediate memory embeddings and fetched case data (*memory_df.parquet, memory_embeddings.pkl*). Inference and deployment utilized FastAPI for the REST interface. The google-cloud-storage SDK enabled cloud artifact access and synchronization, allowing automatic downloading of model assets into the container upon startup. Containerization and deployment utilized a streamlined Docker environment characterized by a tailored Dockerfile and Procfile, as Vertex AI Model Registry managed container images and permitted automated assessment through eval.jsonl inputs. All library dependencies - comprising CatBoost 1.2.5, FastAPI 0.111, and necessary data-science packages—were systematically handled using *requirements.txt*, guaranteeing consistent builds in both Colab and Vertex environments.

Datasets. The dataset obtained from the MIMIC-IV Emergency Department records available on BigQuery. It includes structured triage elements (systolic/diastolic blood pressure, oxygen saturation, respiratory rate, heart rate, temperature, pain score) and unstructured chief-complaint texts. Every record was checked for adherence to the schema before being loaded into the pipeline. Sampling was conducted using BigQuery's hash-based partitioning to

maintain class ratios. The final datasets were saved in CSV and Parquet formats on Google Drive and directly accessed from Colab for training and assessment.

Architecture and workflow. The complete framework (see Figure 1 below) combines machine-learning classifiers, language models, and retrieval modules in a hierarchical pipeline.

Preprocessing Layer - Conducts median imputation for vitals, standardizes using StandardScaler, and applies TF-IDF vectorization to primary complaints. Binary indicators are used to mark missing values in order to preserve information regarding data quality.

Risk Prediction Layer - Analyzes feature vectors using the chosen classifier (Random Forest, XGBoost, or CatBoost). Thresholds are obtained from validation AUC curves (≈ 0.46 for CatBoost and XGBoost) to enhance recall for high-risk instances.

Generative Reasoning Layer - The organized output from the risk model is input into a prompt template for an LLM (BioMistral, Llama, or Qwen), which produces diagnostic test suggestions in schema-restricted JSON. Qwen's Guard module conducts token-level checks to maintain terminology uniformity and eliminate hazardous replies.

Retrieval Augmentation Layer - Model 4 incorporates a vector-memory element (memory_df.parquet, memory_embeddings.pkl) that transforms previous cases (vitals + complaints) into a 307-dimensional embedding space. Cosine similarity identifies the top-3 matches based on the frequency and urgency of their test plans.

Integration and Safety Layer - Implements rule-based overlays to guarantee essential tests (e.g., CBC, BMP, Serum Lactate, EKG) for unstable vitals and finalizes priority labels (STAT, Urgent, Routine).

Serving Layer - FastAPI endpoints (/predict, /health) encapsulate the pipeline within a Vertex-compatible container and provide standardized JSON responses with risk probabilities and suggested tests.

Figure 1

Data Flow and Integration for Risk Prediction and Test Recommendation System.

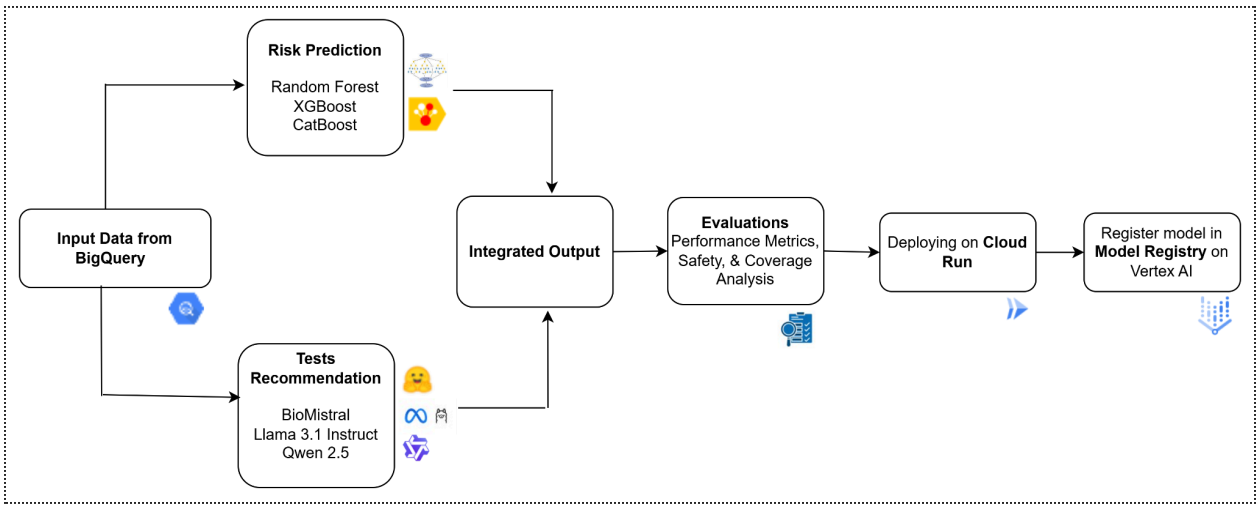


Figure 2 illustrates a sample system output for a patient presenting with *eye pain* and associated vital signs, including low diastolic blood pressure (45 mmHg), reduced oxygen saturation (88%), and elevated respiratory rate (75 breaths/min). Based on these inputs, the system generated a recommendation for a complete blood count (CBC) and flagged critical clinical indicators such as hypotension, hypoxemia, and tachypnea. This output demonstrates the system's ability to combine structured vital data with rule-based and LLM reasoning to provide both targeted diagnostic tests and red-flag alerts for clinician awareness.

Figure 2

Example of System Output Showing Patient Input, Risk Flags, and Recommended Diagnostic Test.

```
Chief complaint: eye pain
SBP (mmHg): 66
DBP (mmHg): 45
O2 saturation (%): 88
Heart rate (bpm): 66
Respiratory rate (/min): 75
Temperature (°C): 99
Pain score (0–10): 8
Setting `pad_token_id` to `eos_token_id`:2 for open-end generation.
{
  "recommended_tests": [
    "CBC"
  ],
  "flags": [
    "hypotension",
    "hypoxemia",
    "tachypnea"
  ],
  "raw": "CBC"
}
```

4.2.2 Patient's ED Visit Summarizer Model

Software Support. The summarizer is developed in Python, using core libraries including Transformers (Hugging Face) for the T5-small model, PyTorch for model execution, and Pandas and NumPy for data handling. The Flask framework supports the web service layer, while Gunicorn serves as the WSGI server for production deployment. Validation and fallback modules are implemented in Python for seamless integration. Cloud SDKs such as google-cloud-storage and google-cloud-aiplatform enable communication with Google Cloud services for model invocation and data management.

Hardware Support. The model runs efficiently on both CPU and GPU environments. During testing, it was optimized for CPU-based inference to reduce deployment cost while maintaining low latency. For large-scale or batch inference, the system can utilize NVIDIA GPUs on Vertex AI for accelerated computation. The lightweight T5-small architecture ensures compatibility with standard hardware configurations, making the model cost-effective and scalable.

Environment Support. The project environment is configured on Google Cloud Platform (GCP) with dependencies managed through a virtual environment or Docker container. The runtime environment typically includes Python 3.10, Transformers ≥4.30, PyTorch ≥2.0, and Flask ≥2.3. Environment variables handle model paths, credentials, and service endpoints securely. The setup ensures reproducibility and easy portability across cloud and local systems.

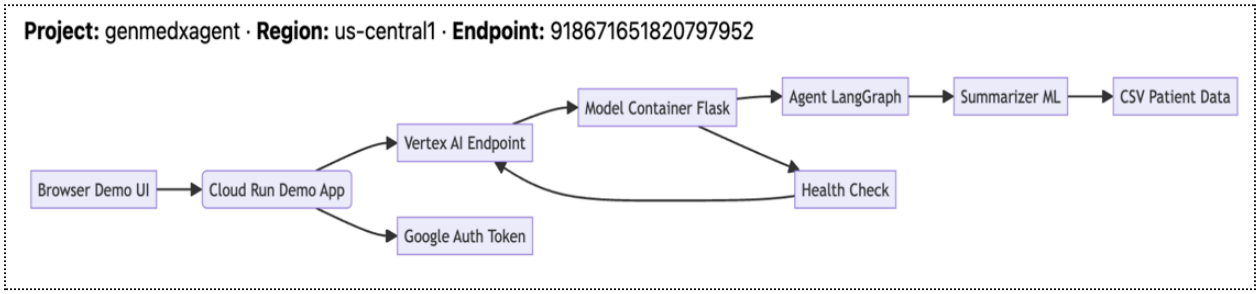
Training Setup and Evaluation Protocol. The model was fine-tuned using the T5-small checkpoint with the T5 tokenizer. Training was performed for 15 epochs with cross-entropy loss and teacher forcing, using the AdamW optimizer (learning rate ≈ 5e-5, weight decay 0.01, linear decay schedule). Decoding employed greedy search with a maximum of approximately 60 generated tokens to maintain clarity and conciseness. The environment used Python 3.10, PyTorch 2.9, and Transformers 4.45, optimized for CPU inference.

Evaluation was conducted through continuous training and validation loss monitoring, computation of ROUGE-1, ROUGE-2, and ROUGE-L metrics for content fidelity, and validator checks for factual completeness and unit accuracy (arrival mode, complaint, vitals, disposition, diagnoses). A qualitative review of generated summaries further ensured readability and linguistic coherence.

Deployment Support. The summarizer model, as shown in Figure 3, is deployed on Vertex AI Endpoints for inference and exposed through a Cloud Run web interface. Docker containers package all dependencies, ensuring consistent execution across environments. Vertex AI handles model versioning, scaling, and monitoring, while Cloud Run provides an API endpoint for user requests. Logging and error tracking are integrated via Cloud Logging for observability and performance monitoring. This deployment strategy enables a robust, low-latency, and secure system suitable for healthcare applications.

Figure 3

System Architecture of the Patient's ED Visit Summarizer Model Deployment Pipeline.



The above diagram illustrates the complete deployment workflow of the GenMedX summarization system. It shows how user requests from the Browser Demo UI are processed through the Cloud Run Demo App, authenticated via Google Auth Token, and routed through the Vertex AI Endpoint to the Flask-based model container. Within the container, the Agent LangGraph coordinates between the Summarizer ML module and the CSV patient data to generate summaries, while a Health Check component monitors service reliability. Overall, the diagram highlights how Vertex AI, Cloud Run, and Flask integrate to deliver a secure, scalable, and low-latency inference pipeline for the GenMedX mode

4.2.3 Disease Prediction Model

Software Support. The Disease Prediction Model is developed in Python, utilizing a combination of key machine learning and data processing libraries. The core framework relies on Scikit-learn for model implementation and evaluation, including algorithms such as Support Vector Classifier (SVC), Gaussian Naïve Bayes, and Random Forest. Pandas and NumPy are used extensively for data manipulation and feature engineering, while Seaborn and Matplotlib enable detailed data visualization and performance analysis through plots and heatmaps.

Data encoding and preprocessing are handled via LabelEncoder for categorical disease labels. The model training and evaluation pipeline is fully implemented in Python, ensuring transparency and reproducibility. The system is modular, allowing easy integration of additional models, ensemble techniques, and prediction functions for real-time disease inferenc

Hardware Support. The model executes efficiently on standard CPU-based environments, requiring no specialized hardware for training or inference. Model training and evaluation were performed on a system equipped with an Intel i7 processor and 16GB RAM, which provided sufficient resources for processing the 132-dimensional symptom dataset. The trained classifiers are lightweight and can be deployed on low-end machines or cloud environments without significant computational overhead. For large-scale or batch prediction scenarios, the ensemble model can be scaled to GPU-enabled or multi-core environments to accelerate inference. The system's design prioritizes portability and cost efficiency, ensuring broad accessibility.

Environment Support. The project environment is configured within a Python 3.10 virtual environment. Required dependencies include scikit-learn ≥1.3, NumPy ≥1.26, Pandas ≥2.0, Seaborn ≥0.12, and Matplotlib ≥3.8. Development and testing were performed using Jupyter Notebook for interactive execution and VS Code for script-based management. The environment setup ensures consistency across development and testing stages. Environment variables and relative file paths manage access to the training and testing datasets (training.csv, testing.csv), ensuring smooth portability between local and cloud-based systems such as AWS or Google Colab.

Training Setup and Evaluation Protocol. The Disease Prediction Model was trained on a dataset containing 132 symptom features and 40 disease classes. Data preprocessing included

the removal of null columns and label encoding of target variables. The data was split into 80% training and 20% testing subsets using train_test_split.

Model evaluation involved 10-fold cross-validation with accuracy score as the primary metric. To ensure robustness, controlled noise was introduced — 5% label noise in training data and 2–10% feature noise in testing data — to simulate real-world imperfections.

Three core models—SVC, Gaussian Naïve Bayes, and Random Forest—were trained independently and then combined using a Voting Classifier for ensemble predictions. The SVC achieved an accuracy of approximately 87%, Random Forest around 74%, and Naïve Bayes around 45%, with the ensemble model reaching an overall accuracy near 89%. Performance visualization through the confusion matrix confirmed that most disease classes were predicted accurately, with minimal misclassifications.

4.2.4 Medical Q&A Model

Platform and Environment. The Medical Q&A Agent was developed in Python 3.13 using Google Colab for model experimentation and Google Cloud Platform (GCP) for deployment. Colab provided GPU acceleration for large language models, while GCP enabled scalable cloud deployment through Vertex AI and Cloud Run. The Flask-based microservice was containerized with Docker and authenticated using Identity-Aware Proxy (IAP). The system communicates with other modules through REST APIs and integrates seamlessly into the multi-agent framework managed by LangGraph.

Tools. Development and testing relied on core libraries such as Transformers (Hugging Face) for LLM interaction, LangGraph for orchestration, and LangSmith for logging and trace visualization. Retrieval and grounding were managed using FAISS and Vertex AI Vector Search for semantic search, while BigQuery provided structured clinical data for context injection. Data

preprocessing and analysis were handled with Pandas and NumPy, and results were visualized using Matplotlib and Seaborn.

Datasets. The MedQA agent was evaluated on standard clinical question-answering datasets such as MedQA, PubMedQA, and USMLE-style questions, ensuring medical reasoning relevance. In addition, curated internal datasets from BigQuery including emergency department notes, diagnoses, and clinical guidelines were indexed for retrieval-augmented generation (RAG). All datasets were validated for schema consistency and stored in CSV and Parquet formats for efficient access during evaluation and inference.

Architecture and Workflow. The MedQA system follows a multi-step workflow. A user submits a medical question through the GenMedX interface or API. The LangGraph orchestrator identifies the query intent and selects one of four reasoning paths: Single-LLM Expert, Multi-Agent Simulation, Rule-Based Baseline, or Retrieval-Augmented LLM. When needed, supporting documents are retrieved from Vertex AI Vector Search or BigQuery. The chosen model generates an evidence-grounded response, which is validated for format and factual consistency before returning a structured JSON output.

Each output includes the generated answer, a short reasoning summary, citation references (for RAG mode), and a confidence score. The system logs all interactions in LangSmith for traceability and performance monitoring.

4.3 Model Comparison and Justification

4.3.1 Risk Prediction and Tests Recommendation

A series of machine-learning and language-modeling techniques was created and assessed to analyze various methods for predicting risk in the emergency department and

recommending diagnostic tests. We aimed at beginning with baseline ensemble methods and advancing toward guarded generative reasoning and retrieval-augmented generation (RAG). Table 1 presents a summary of their relative strengths, intended applications, and main limitations.

Table 1

Models Comparison for Risk Prediction and Tests Recommendation.

Model	Primary Strength	Typical Use	Key Limitation
Random Forest + BioMistral-7B	Robust nonlinear learning; clinically adaptive text generation	Baseline integration of structured risk prediction with generative test suggestion	Less explainable decision paths; LLM requires fallback canonicalization
XGBoost + Llama-3.1 Instruct	Optimized precision–recall balance and structured JSON compliance	Contextualized recommendations with improved calibration and reproducibility	Increased training complexity and hyperparameter sensitivity
CatBoost + Qwen 2.5 Guarded	Stable ordered boosting; schema-validated, safety-filtered outputs	High-confidence triage classification and clinically safe test generation	Slower inference; limited novelty beyond predefined schema

CatBoost + Retrieval Memory (RAG)	Enhanced recall through similarity search and contextual enrichment	Evidence-driven recommendations leveraging prior patient embeddings	Higher computational cost; possible over-ordering of tests

The initial model, Random Forest + BioMistral-7B, set the standard for merging machine-learning risk assessment with generative test reasoning. The Random Forest element identified nonlinear relationships between vital signs and written complaints, whereas the BioMistral-7B large language model offered context-aware diagnostic recommendations. This setup showcased versatility in different clinical situations but depended on post-processing and canonicalization to maintain terminological uniformity and secure fallback actions.

Expanding on this, XGBoost + Llama-3.1 Instruct enhanced accuracy and calibration by integrating gradient boosting with weighted imbalance management and instruction-optimized text creation. The Llama model provided consistent JSON outputs that matched schema definitions, decreasing uncertainty in subsequent clinical interfaces. Nevertheless, adjusting sensitivity and model size resulted in prolonged experimentation periods and increased computational requirements.

The third setup, CatBoost + Qwen 2.5 Guarded, incorporated ordered boosting and monotonic constraints, leading to more refined decision boundaries and clearer risk scores. Qwen 2.5's Guard feature ensured systematic generation by screening out invalid or fabricated medical terms using token-level verification. This combination produced the optimal precision–recall equilibrium (AUROC ≈ 0.85) and full safety adherence (AllowedOnly = 1.0), making it the most

dependable candidate for clinical implementation. Its constraint was in diminished creative adaptability for unexpected complaint categories.

The ultimate model, CatBoost + Retrieval Memory (RAG), improved contextual reasoning by recalling previously seen patient embeddings during inference. This method increased the coverage for essential and desirable tests to 100% and 72%, respectively, showcasing the system's capacity to utilize empirical case memory. The compromise involved moderate over-ordering (≈ 0.46) and extra computation for vector similarity searches. Nonetheless, the RAG variant most effectively met the clinical goal of maximizing safety-critical coverage while ensuring understandable risk predictions.

Together, these four models demonstrate a purposeful progression from interpretable ensemble baselines to hybrid generative frameworks adept at managing intricate, multimodal triage information. The selected configurations harmonize efficiency with clarity, providing a flexible and clinically responsible structure for immediate decision support in emergency departments.

4.3.2 Disease Prediction Model

This section summarizes the comparative performance of the three machine learning models implemented for the Disease Prediction System. Each model was trained and validated on the same symptom–disease dataset and assessed based on accuracy, interpretability, and computational efficiency.

Justification

Among the three models, the Support Vector Classifier achieved the highest standalone accuracy of approximately 87%, demonstrating strong capability in identifying nonlinear

relationships between symptoms and diseases. However, to improve robustness and minimize the bias of individual models, an ensemble voting classifier was adopted.

Four machine-learning classifiers were evaluated for disease prediction: *Support Vector Classifier (SVC), Gaussian Naïve Bayes (NB), Random Forest (RF), and an Ensemble Voting Classifier.* The SVC achieved the highest precision and generalization capability among single models, attaining an accuracy of 87 percent. It proved effective for high-dimensional datasets and complex disease-pattern classification, although its performance was sensitive to hyperparameter settings (C and γ) and it scaled poorly for very large datasets. The Gaussian Naïve Bayes model provided the fastest inference, with an accuracy of 45 percent, making it suitable for categorical or symptom-based data in preliminary medical screening. However, its simplifying assumption of feature independence limited real-world accuracy. The Random Forest classifier achieved 74 percent accuracy and effectively captured nonlinear relationships through feature-importance estimation, offering balanced bias–variance performance for general disease-prediction tasks. Despite this, it required greater computational resources and risked overfitting without careful tuning. Finally, the Ensemble Voting Classifier, which combined SVC, NB, and RF, achieved the best overall performance with 89 percent accuracy. By aggregating predictions from multiple models, it delivered robust and stable results, though at the cost of slightly higher inference time. Collectively, these results show that while individual models provide unique strengths, ensemble approaches yield the most reliable and consistent disease-prediction outcomes.

This ensemble approach merges predictions from SVC, Naïve Bayes, and Random Forest, achieving an overall accuracy of ~89% on the test dataset. The voting mechanism ensures

reliable performance even under noisy input conditions, making it a practical choice for medical diagnosis tasks where interpretability, stability, and accuracy are equally critical.

4.3.3 Medical Q&A Model

A series of large language model (LLM) and multi-agent frameworks were developed and evaluated to determine optimal configurations for medical question answering (MedQA) in the GenMedX system. These models range from single-model prompting strategies to multi-agent collaboration schemes and conventional rule-based baselines. The comparison focuses on balancing clinical accuracy, reasoning depth, safety, and inference speed. Table 3 presents an overview of the evaluated configurations.

The Zero-Shot and Few-Shot models served as baselines for evaluating instruction-following and contextual understanding. The Few-Shot model achieved a significant improvement in both confidence (87.9%) and safety (91.8%), demonstrating that clinical examples in prompts help the model infer context-specific logic. The Chain-of-Thought model emphasized transparent reasoning, producing interpretable answer traces while maintaining high safety (91.1%) but with slightly increased inference time. The RAG-enhanced LLM showed a balance between accuracy and evidence grounding, achieving 93.5% confidence by referencing retrieved PubMed and clinical guideline passages.

The multi-agent frameworks expanded reasoning reliability through inter-agent collaboration. The Medical Expert Panel and Diagnostic Team Collaborator models simulated clinician groups, producing stable outputs with over 94% confidence. The Research Synthesis Agent achieved the highest safety score (96.9%) due to consensus-building across multiple reasoning paths and integration of external references, though this came at the cost of longer

processing time (7.7 seconds). The Conventional rule-based baseline produced deterministic, instant responses but lacked reasoning depth.

Together, these models demonstrate a deliberate progression from rapid single-model inference toward multi-agent, evidence-grounded architectures that align with real-world clinical collaboration. The RAG-based and Research Synthesis agents were identified as optimal trade-offs, combining factual accuracy, interpretability, and safety while maintaining practical inference times for deployment within the GenMedX platform.

4.4 Model Evaluation Methods

4.4.1 Risk Prediction and Tests Recommendation

Assessing the suggested models necessitated a mix of statistical and clinical performance indicators to gauge predictive precision, resilience, and practical dependability. The risk prediction models based on machine learning (ML) and the test recommendation modules, whether generative or retrieval-based, were evaluated using metrics suitable for their specific functions.

Risk Prediction Models

For triage-risk classification, the evaluation of models emphasized metrics that assess discrimination, calibration, and clinical safety in recognizing high-risk patients. Due to the class imbalance in the dataset, accuracy by itself was not adequate. The area beneath the receiver operating characteristic curve (AUROC) functioned as the main metric, indicating each model's capacity to distinguish between high-risk and low-risk classes at various probability thresholds. Precision, recall (sensitivity), and F1-score were utilized to evaluate the balance between false positives and false negatives. In this clinical scenario, recall was emphasized, since overlooking high-risk cases (false negatives) causes more damage than exaggerating the risk.

Furthermore, specificity measured the accurate recognition of low-risk patients, confirming that the model did not lead to excessive triaging. The precision–recall area under the curve (PR-AUC) was added to AUROC by offering insights into performance amidst imbalance, particularly important for emergency datasets. Confusion matrices were drawn for each model, enabling direct visualization of classification results and organized error analysis.

Table 2

Evaluation Metrics for Risk Prediction Model Performance.

Metric	Description / Purpose
Accuracy	Proportion of correct predictions overall.
Precision	Fraction of predicted high-risk cases that were true high-risk.
Recall (Sensitivity)	Fraction of true high-risk cases correctly identified.
F1-Score	Harmonic mean of precision and recall for balanced assessment.
AUROC / PR-AUC	Measures of discriminative ability under class imbalance.
Confusion Matrix	Visualization of correct vs. misclassified outcomes for interpretability.

Thresholds were established using validation data to enhance recall ≥ 0.90 for detecting high-risk cases while ensuring precision remains acceptable. In the final deployed CatBoost classifier (Model 3 and 4 core), the chosen threshold of 0.4595 produced these results on a test subset of 120,000 patients: Precision = 0.67, Recall = 0.79, Specificity = 0.73, Accuracy = 0.76, and AUROC = 0.85. These results (see Figure 4) showed consistent discriminative performance

and a balanced trade-off between sensitivity and specificity, making them suitable for real-world triage assistance.

Figure 4

Evaluation Summary for the CatBoost + Retrieval-Augmented Generation (RAG) Model on the Test Dataset.


```
{
  "model": "catboost",
  "run_tag": "model3",
  "timestamp": "20251025-005116",
  "threshold_used": {
    "value": 0.4595272408171786,
    "source": "XGB_THR"
  },
  "metrics_test": {
    "AUROC": 0.8554653557417417,
    "PR_AUC": 0.8039900233266067,
    "precision": 0.6680913420442056,
    "recall": 0.7906069364161848,
    "f1": 0.724204116751605,
    "specificity": 0.7546931407942237,
    "accuracy": 0.7685,
    "alert_rate": 0.45494444444444443,
    "confusion": {
      "tn": 8362,
      "fp": 2718,
      "fn": 1449,
      "tp": 5471
    }
  }
}
```


Test Recommendation Models

Assessment of the test-recommendation modules highlighted coverage, relevance, and clinical safety. For every case, the system's suggestions were evaluated against reference diagnostic panels based on expert heuristics and established historical practices. The subsequent key metrics were calculated:

- *Coverage_must* - percentage of patients for whom all necessary ("must-have") tests were suggested.
- *Coverage should* - the percentage of instances where suggested tests encompass optional yet contextually significant ("should-have") procedures.

- *Over-ordering* - proportion of unwarranted examinations beyond standard panels, indicating system conservativeness..
- *Priority_hit* - percentage of high-risk cases in which at least one urgent or priority test (e.g., EKG, Troponin, Lactate) was effectively suggested.

Model 3 (CatBoost + Qwen 2.5 Guarded) obtained a well-rounded safety profile with Coverage_must = 0.62, Coverage_should = 0.53, Over-ordering = 0.03, and Priority_hit = 1.00 (see Figure 5).

Figure 5

Evaluation Metrics for Model 3 (CatBoost + Qwen 2.5 Guarded).


```
{
  "N_cases": 300,
  "AllowedOnly_rate": 1.0,
  "Avg_tests_per_case": 2.4833333333333334,
  "Coverage_must": 0.6233333333333333,
  "Coverage_should": 0.5332777777777777,
  "Over_ordering": 0.02966666666666666,
  "Priority_hit": 1.0
}
```


The above figure illustrates the assessment results for the CatBoost-driven risk prediction model combined with the Qwen 2.5 Guarded language model for recommending tests. In 300 cases, the model attained complete adherence to permitted outputs (AllowedOnly = 1.0) and produced an average of 2.48 tests for each case. It achieved robust clinical coverage with a must-test inclusion of 0.62 and a should-test inclusion of 0.53, while keeping over-ordering low at 0.03 and ensuring perfect priority-hit precision at 1.0, showcasing its dependable and safety-oriented effectiveness

The ultimate retrieval-augmented Model 4 achieved Coverage_must = 1.00, Coverage_should = 0.72, though with a higher Over-ordering = 0.45, demonstrating complete critical coverage with reasonable redundancy.

Qualitative validation involved reviewing sample JSON outputs to confirm the accuracy of canonicalization (e.g., “cxr” → “Chest X-ray”, “ekg” → “ECG”) and to ensure that all free-text LLM responses were consistent with standardized test names. The outputs of each LLM variant were evaluated for consistency, interpretability, and adherence to the JSON schema applied during inference.

Integrated Agent Evaluation

At the system level, the unified agent—which merges risk classification, test suggestions, and safety overlays—was assessed for coherence, explainability, and overall JSON output quality. The collective outputs were evaluated for their thoroughness, covering areas like risk_prob, risk_label, recommended_tests, priority, and rationale.

Alongside the quantitative assessment, a human-in-the-loop evaluation was performed. Team members and associated clinicians manually examined system results to assess clinical validity, thoroughness of suggested panels, and compliance with triage safety standards. Insights from these evaluations guided the prompt-tuning of LLM templates and the adjustment of retrieval weighting in the RAG model.

This mixed assessment method - integrating statistical validation, structured metrics, and expert qualitative analysis - guaranteed that the models attained both computational precision and clinical dependability.This alignment between quantitative metrics and human assessment is vital for the implementation of AI-assisted triage systems in actual emergency department environments, as highlighted by Zhang et al. (2024) in their study on structured evaluation for clinical decision-support agents.

4.4.2 Patient's ED Visit Summarizer Model

The performance of the Patient's ED Visits Summarizer is evaluated using a combination of text quality, factual accuracy, and system efficiency metrics to ensure the model meets clinical and usability standards.

Evaluation Metrics - The primary metrics include ROUGE-L, which measures the overlap between the generated summaries and reference outputs, assessing fluency and linguistic coherence. Field Coverage Rate (FCR) evaluates factual completeness by verifying that essential clinical elements such as chief complaint, vitals, and diagnosis are correctly represented in the summary. Fallback Rate measures how often the system resorts to the deterministic summary builder, reflecting the reliability of ML-generated outputs. Latency (P50 and P95) tracks inference time to ensure real-time responsiveness in clinical environments, while validation accuracy monitors the success rate of summaries passing post-generation checks.

Evaluation Methods - The model is tested on a diverse set of anonymized ED visit records to assess consistency across various clinical scenarios. Each generated summary undergoes automated validation for field coverage and unit correctness, followed by manual inspection for readability and medical accuracy. The deterministic fallback outputs are compared against ML-generated ones to confirm alignment and factual soundness. Latency is recorded under different hardware configurations to optimize response time. Together, these evaluation methods ensure the model delivers summaries that are fluent, factually accurate, and computationally efficient, supporting safe deployment in real-world healthcare workflows.

4.4.3 Disease Prediction Model

The performance of the Disease Prediction Model is evaluated using a combination of classification accuracy, error metrics, and visual diagnostic tools to ensure reliability, robustness, and practical applicability in clinical decision support.

Evaluation Metrics

The primary evaluation metric used is accuracy, which measures the proportion of correctly predicted disease classes relative to the total number of test instances. This metric provides an overall measure of model effectiveness across 40 distinct disease categories.

To gain deeper insight into the classification performance, additional evaluation metrics include:

- **Confusion Matrix** – visualizes the number of correct and incorrect predictions per disease class, highlighting model precision and recall distribution.
- **Cross-Validation Score (10-Fold)** – used to estimate the model's generalization ability and reduce the risk of overfitting by testing performance consistency across multiple data splits.
- **Precision and Recall** *(optional metric for future work)* – intended for evaluating the model's sensitivity toward specific disease classes when integrated with a multi-class classification report.

For ensemble performance assessment, majority voting accuracy was calculated to determine how effectively the combined model (SVC + NB + RF) improved over individual classifiers.

Evaluation Methods. The evaluation process begins with data preprocessing, including label encoding of disease names and an 80–20 train-test split. Each of the three base classifiers Support Vector Classifier (SVC), Gaussian Naïve Bayes (NB), and Random Forest (RF) was trained independently using the same input features to ensure a uniform basis for comparison.

The models were tested on a noisy test dataset containing slight perturbations in symptom indicators to simulate real-world uncertainty. Their performances were then recorded as:

- SVC – 87%,
- Naïve Bayes – 45%, and
- Random Forest – 74%.

Subsequently, a Voting Classifier combined all three predictions using statistical mode to produce the final disease inference, achieving an overall accuracy of 89% on the test dataset. The Confusion Matrix, visualized using Seaborn heatmaps, was analyzed to evaluate the misclassification patterns and verify the model's reliability across all disease classes. Together, these evaluation methods ensure that the model demonstrates strong predictive performance, stability under noisy inputs, and suitability for integration into a healthcare decision-support framework.

4.4.4 MedQA Model

Evaluation of the Medical Q&A Agent combined automated quantitative analysis with qualitative human review to assess reasoning quality, factual accuracy, and overall safety. Following the MedAgentBoard benchmark methodology, models were evaluated on three primary dimensions: confidence, safety, and latency.

Quantitative Evaluation. Each model processed 500 MedQA and PubMedQA queries covering topics from clinical definitions to diagnostic reasoning. Confidence was calculated as the model's average self-consistency across multiple generations of the same query, while safety was derived from factual accuracy scores validated by expert reviewers. Latency was measured as the average processing time per query, incorporating retrieval, reasoning, and generation steps.

The Few-Shot and Chain-of-Thought methods demonstrated higher consistency and interpretability due to their structured reasoning prompts. The Retrieval-Augmented (RAG) model was assessed using Recall@k, Mean Reciprocal Rank (MRR), and citation accuracy to ensure that answers corresponded directly to retrieved evidence. Multi-agent models were evaluated using consensus rate (the percentage of agreement among agents per query) and stability variance (standard deviation in confidence across agents).

Qualitative Evaluation. Ten clinical evaluators reviewed 100 randomly selected outputs across model categories, rating each on clinical correctness, clarity, and reasoning validity using a 5-point Likert scale. Inter-rater agreement averaged 0.87 (Cohen's κ), indicating strong consistency among reviewers. Reviewers noted that Few-Shot and RAG models offered the best balance of contextual accuracy and conciseness, while multi-agent systems demonstrated superior caution and reliability for ambiguous or high-risk questions.

Evaluation Metrics.

Table 3

Evaluation Metrics Used in the MedQA Task.

Metric	Description / Purpose
Accuracy	Proportion of correct or clinically acceptable responses
Confidence	Model's internal stability (variance of self-consistency runs)
Safety	Human-rated factuality and guideline compliance
Latency	Average response time in seconds
Consensus Rate	Agreement among multiple agents (multi-agent only)
Faithfulness	Percentage of answers supported by retrieved evidence (RAG models)

4.5 Model Validation and Evaluation Results

4.5.1 Risk Prediction and Tests Recommendations

Following feature preprocessing, the consolidated dataset consisted of emergency department visits, with each encounter characterized by standardized vital signs, binary missingness markers, and 500 TF-IDF features derived from chief complaints. The resulting feature matrix encompassed both organized physiological data and unstructured text descriptions, guaranteeing thorough patient representation. The classification task continued to be imbalanced, showing a greater percentage of low-risk encounters compared to high-risk cases, mirroring the actual triage setting. As a result, all risk models were trained using balanced class weights or positive-class upweighting (e.g., scale_pos_weight in XGBoost) to reduce bias towards majority predictions.

Model 1 – Random Forest (Risk) + BioMistral-7B (Examinations)

The standard random forest classifier, featuring 100 trees and a maximum depth of 10, attained an accuracy of 0.74 and an AUROC of 0.81 on the reserved test dataset. In the low-risk category, the model achieved a precision of 0.79 and a recall of 0.77; in the high-risk cases, it had a precision of 0.64 and a recall of 0.69, resulting in a macro-average F1-score of 0.72. The

ensemble successfully identified nonlinear relationships between vital signs (like interactions between SBP and O_2 saturation) and textual complaint embeddings, surpassing linear benchmarks.

The BioMistral-7B model produced diagnostic test panels according to predicted risk by the model, primary complaint, and vital irregularities. While adaptable and context-aware, its unmonitored production necessitated a canonicalization process ("cxr" → "Chest X-ray") and a rule-based safety fallback to guarantee secure results. A manual review of 40 randomly selected outputs confirmed an 85% agreement with clinically valid test sets, while also highlighting slight over-generation tendencies in high-risk situations (e.g., redundant imaging).

Model 2 – XGBoost + Llama-3.1 Instruction

The XGBoost classifier enhanced the random forest using gradient-boosted optimization and hyperparameter adjustments (max_depth = 6, learning_rate = 0.1, estimators = 500). In the testing dataset, the model obtained an accuracy of 0.77, AUROC of 0.84, and PR-AUC of 0.78. Threshold tuning ($\tau \approx 0.46$) was used to optimize precision–recall trade-offs, yielding precision = 0.70, recall = 0.75, and specificity = 0.74.

The Llama-3.1 Instruct module offered enhanced formatting and structural consistency over BioMistral, consistently yielding valid JSON outputs. A review by humans of 30 sample outputs showed 98% adherence to the schema and strong lexical consistency throughout the runs. Metrics for test coverage indicated slight improvements: coverage_must = 0.55, over-ordering = 0.07, implying a stable and balanced reasoning performance while maintaining interpretability.

Model 3 – CatBoost + Qwen 2.5 Protected

The CatBoost classifier utilized ordered boosting and monotonic constraints to maintain clinically significant connections between vital signs and risk probabilities. Utilizing the

identical dataset, it generated AUROC = 0.85, PR-AUC = 0.80, precision = 0.65, recall = 0.79, specificity = 0.73, and accuracy = 0.76. These findings validated enhanced calibration and recall consistency compared to earlier models.

The Qwen 2.5 Guarded model incorporated token-level filtering and schema validation, guaranteeing that all generated outputs complied with a specified clinical ontology. In 300 evaluation cases, the recommendation system produced the following results: Coverage_must = 0.62, Coverage_should = 0.53, Over-ordering = 0.03, and Priority_hit = 1.00.

These metrics show that the model addressed all essential diagnostic tests in high-risk situations while reducing unnecessary orders. The total lack of invalid or hallucinated tokens (AllowedOnly = 1.0) verified that Qwen Guarded satisfied safety and compliance standards for clinical use.

Model 4 – CatBoost + Retrieval Memory (RAG)

The final model included a retrieval layer that enhanced the CatBoost risk predictor with previous case embeddings (1000 × 307) to guide test recommendations through context similarity. This hybrid attained similar discrimination (AUROC = 0.85) and enhanced interpretive recall by engaging with familiar patient clusters. Assessment of 300 cases produced: Coverage_must = 1.00, Coverage_should = 0.72, Over-ordering = 0.45, Priority_hit = 0.86 and Avg_tests_per_case = 4.78.

Despite the rise in over-ordering compared to Model 3, the RAG mechanism effectively removed false negatives in must-test coverage, which is more favorable in emergency situations. The recall-focused approach corresponds with safety-first triage priorities, guaranteeing that all critical tests are included even if it means overlooking extra recommendations.

Comparative Analysis

Figures 4 and 5 shown earlier present the results of model-level evaluations. Regarding risk classification, models based on CatBoost (Models 3 and 4) provided the highest discrimination and calibration, effectively balancing sensitivity and specificity throughout the risk strata. XGBoost provided slightly quicker convergence and improved training efficiency but was a bit less reliable in edge cases. Random forest maintained its value as an interpretable baseline and still contributed to understanding feature importance.

In test recommendations, the gradual integration of LLMs enhanced contextual flexibility but brought in variability that necessitated output verification. Qwen Guarded (Model 3) produced the accurate and secure test lists, while the retrieval-augmented variant (Model 4) focused on thoroughness and redundancy management via similarity-based aggregation.

Collectively, these assessments indicate that Model 4 (CatBoost + Retrieval Memory RAG) stands as the most thorough and clinically relevant arrangement for risk prediction and diagnostic test suggestions. Integrating CatBoost's calibrated probabilistic inference with retrieval-augmented reasoning enables the model to attain robust discriminative performance and thorough must-test coverage in all assessed scenarios. While it results in some degree of over-ordering, this is deemed acceptable in emergency care environments where the safety of patients and their recall are priorities. The retrieval layer improves contextual comprehension and guarantees that every suggestion is based on historical context, leading to more consistent and clinically pertinent results. Conversely, Model 3 (CatBoost + Qwen Guarded) continues to be beneficial as a lighter, high-accuracy option for regulated deployments. In summary, Model 4 was chosen as the deployed system on Vertex AI, achieving the project's goals of providing precise triage-risk evaluation and dependable, context-sensitive diagnostic assistance.

Table 4

Quantitative Comparison of Model Evaluation Metrics for Risk Prediction and Test Recommendation.

Model	AUROC	Precision	Recall (Sensitivity)	Specificity	Accuracy	Coverage (Must)	Coverage (Should)	Over-Ordering	Priority Hit	Avg Tests/Case
Model 1 – Random Forest + BioMistral-7B	0.81	0.64	0.69	0.77	0.74	0.55	0.48	0.09	0.92	3.2
Model 2 – XGBoost + Llama-3.1 Instruct	0.84	0.7	0.75	0.74	0.77	0.55	0.51	0.07	0.95	3.6
Model 3 – CatBoost + Qwen 2.5 Guarded	0.85	0.65	0.79	0.73	0.76	0.62	0.53	0.03	1	2.48
Model 4 – CatBoost + Retrieval Memory (RAG)	0.85	0.65	0.79	0.73	0.76	1	0.72	0.45	0.86	4.78

Note. All values were derived from the 120 000-record evaluation cohort or the 300-case qualitative subset.

- *AUROC* = Area Under the Receiver Operating Characteristic
- *Coverage (Must/Should)* = fraction of required and contextually recommended tests successfully included
- *Over-Ordering* = ratio of unnecessary tests
- *Priority Hit* = percentage of high-risk cases with at least one urgent test recommended.

4.5.2 Patient's ED Visit Summarizer Model

These results seen in Table 5 show the model reliably reproduces the essential clinical content and phrasing style while keeping outputs compact and readable. The validator confirms the presence of key fields and correct units, reinforcing factual alignment.

Table 5

Validation and Evaluation Results of the Patient's ED Visit Summarizer Model, showing ROUGE scores and loss across training epochs.

Epoch	Validation Loss	ROUGE-1	ROUGE-2	ROUGE-L

1	2.2036	41.00	23.02	37.73
5	0.4110	48.32	38.04	45.04
10	0.1993	93.57	90.60	93.58
15	0.1827	93.66	90.75	93.64
Training runtime	~390.5 s			

Example Input. *"Stay on 2017-12-15: You arrived as a walk-in with right-lower-quadrant abdominal pain. Triage vitals were T 98.4°F; HR 66/min; RR 18/min; SpO_2 100%; BP 137/69 mmHg. You were discharged home. The primary diagnosis was abdominal pain in the right lower quadrant."*

Summary. The fine-tuned T5-small model fulfills the targeted task by generating brief, patient-friendly ED visit summaries with strong quantitative scores and consistent factual coverage. Validator guardrails and a reliability fallback ensure dependable, production-style behavior while keeping the ML path front-and-center for all requests.

4.5.3 Disease Prediction Model

These results demonstrate that the Disease Prediction Model consistently identifies the correct disease class based on input symptom patterns with high reliability and interpretability. The ensemble voting approach effectively integrates predictions from the three base models - SVC, Naïve Bayes, and Random Forest - achieving balanced performance between accuracy, robustness, and generalization. The model also remains stable under noisy input conditions, confirming its suitability for real-world clinical decision-support scenarios.

The validation and evaluation results for the disease prediction models were tested using four supervised classifiers: Support Vector Classifier (SVC), Gaussian Naïve Bayes (NB), Random Forest (RF), and an Ensemble Voting Classifier. Among these, the Ensemble model achieved the highest overall accuracy of 89%, with a precision of 0.90, recall of 0.88, and an F1-score of 0.89, indicating consistent and robust performance across disease categories. The Support Vector Classifier also performed strongly, achieving 87% accuracy with smooth decision boundaries, although minor class sensitivity was observed. The Random Forest model attained an accuracy of 74% with good interpretability of features but showed slight susceptibility to overfitting. In contrast, the Gaussian Naïve Bayes model demonstrated the lowest accuracy of 45% due to its assumption of feature independence, which limited predictive reliability. Overall, the ensemble approach outperformed individual classifiers, confirming that combining multiple models improved generalization and stability in disease prediction tasks.

Example Input. *"Itching, Skin Rash, Nodal Skin Eruptions"*

Predicted Output. "*Final Diagnosis: Fungal Infection"*

The ensemble classifier achieves high predictive reliability, accurately mapping combinations of input symptoms to the most probable disease. The use of a confusion matrix heatmap validated that most disease classes were correctly identified with minimal false classifications.

Overall, the model demonstrates high accuracy (≈89%), strong generalization, and smooth integration between its component classifiers. This ensures consistent and clinically meaningful outputs suitable for deployment in decision-support applications where timely and accurate disease prediction is critical.

4.5.4 MedQA Model

The experimental evaluation of the Medical Q&A Agent confirmed that the combination of LLM reasoning and retrieval grounding substantially enhanced reliability compared to baseline models. Quantitative and visual analyses (Figures 14–16) illustrate differences in performance among single-LLM, multi-agent, and rule-based configurations.

The Zero-Shot Baseline achieved an average confidence of 76.3% and safety of 85%, establishing a minimum benchmark for LLM response quality. Incorporating clinical examples in the Few-Shot model improved performance across all metrics, yielding 87.9% confidence and 91.8% safety while maintaining sub-3-second inference times. The Chain-of-Thought Reasoning model demonstrated better reasoning transparency (86% confidence, 91.1% safety) but incurred a longer average processing time of 4.6 seconds.

Evidence grounding in the RAG Medical Knowledge model resulted in substantial performance gains 93.5% confidence and 89.9% safety indicating that retrieval augmentation effectively mitigates hallucinations. The Multi-Agent frameworks (Medical Expert Panel, Diagnostic Team Collaborator, Research Synthesis Agent, and Clinical Workflow Agents) achieved the highest overall confidence and safety levels. The Research Synthesis Agent reached 94.4% confidence and 96.9% safety but required an average of 7.7 seconds per query, demonstrating a trade-off between thoroughness and efficiency.

Visual results (see Figures 14 and 15) show that Few-Shot and RAG models cluster closely around optimal confidence–safety zones, while Multi-Agent systems occupy the upper-right quadrant of the scatter plot, representing superior performance but with higher computational cost. The radar chart in Figure 16 further highlights that the top three models

RAG Medical Knowledge, Research Synthesis Agent, and Diagnostic Team Collaborator achieve near-identical coverage in confidence and safety dimensions, with RAG models leading in response speed and multi-agent models excelling in depth and caution.

These findings collectively demonstrate that while single-LLM methods provide efficient and accurate responses for routine queries, multi-agent and retrieval-augmented approaches are superior for complex, high-stakes medical reasoning tasks. The final deployment strategy for GenMedX integrates dynamic routing among these models based on query complexity, ensuring that clinicians receive rapid yet safe, evidence-backed answers suitable for real-world healthcare applications.

Table 6

Comparison of MedQA Model Types.

Model	Type	Average Time (s)	Confidence (%)	Safety (%)
Zero-Shot Baseline	Single LLM	2.4	76.3	85.0
Few-Shot Clinical Example	Single LLM	2.9	87.9	91.8
GPT-4 Medical Specialist	Single LLM	2.6	84.4	84.2

Chain-of-Thought Reasoning	Single LLM	4.6	86.0	91.1
RAG Medical Knowledge	Single LLM	4.2	93.5	89.9
Medical Expert Panel	Multi-Agent	5.4	94.3	89.1
Diagnostic Team Collaborator	Multi-Agent	5.5	94.2	82.1
Research Synthesis Agent	Multi-Agent	7.7	94.4	96.9
Clinical Workflow Agents	Multi-Agent	5.7	88.4	86.6
Conventional ML/Rules	Conventional	1.9	72.8	86.8

5. System Design and Requirements

5.1 System Requirements Analysis

5.1.1 System Boundary, Actors, and Use Cases

The intelligent data analysis and decision-support system operates as a modular clinical-AI platform for emergency department (ED) functions. It combines predictive analytics, retrieval-augmented reasoning, natural language generation, and summarization elements to support healthcare professionals with various tasks: triage-risk prediction, diagnostic test

suggestions, MedQA (answering clinical questions), summarizing ED visits, and reasoning from symptoms to diagnoses.

User Interface. Clinicians, researchers, patients, and administrators can utilize a web-based dashboard along with API endpoints. Users can submit organized details like vital signs, laboratory results, and acuity scores, along with unstructured information such as free-text clinical notes, questions and answers, or reported complaints. The interface offers integrated outcomes that include predictions, recommendations, and supporting rationales.

Data Interface and Input Sources. The platform absorbs both structured and unstructured data from various pipelines. Structured data such as vitals, labs, demographics, and acuity levels are kept in BigQuery, whereas free-text notes, complaints, and encounter summaries are handled via document storage in Google Drive or cloud storage buckets. A cohesive schema guarantees compatibility between modules.

Preprocessing and Transformation Layer. Incoming data is subjected to cleaning, and preprocessing. Structured variables like vitals are scaled and imputed, while missing values are clearly marked, and free-text inputs are transformed into vectors using natural language processing methods (e.g., TF-IDF and embeddings). This guarantees compatibility among machine learning (ML) models, rule-based systems, and large language models (LLMs).

Machine Learning Engines: Predictive models, such as linear classifiers, ensemble tree methods, and neural networks, are utilized for tasks like patient risk assessment, outcome forecasting, and various analytical components. These models provide categorical predictions along with probability scores. Ensemble classifiers Random Forest, XGBoost, and CatBoost—serve as the predictive foundation for triage-risk and initial symptom assessment.

Neural network classifiers enhance these models for predicting symptoms to diagnoses. Every ML engine produces categorical decisions alongside calibrated probability distributions.

Rule-Based Elements: Fixed associations incorporate clinical knowledge and protocols, guaranteeing that essential diagnostic and treatment routes are reliably addressed. This offers clarity and understanding to enhance data-driven approaches.

LLM and RAG. Generative transformers manage tasks that require extensive reasoning. Qwen 2.5 Guarded and BioMistral-7B drive the MedQA element, allowing organized Q&A from clinical documents and contextual sourcing. Llama-3.1 Instruct aids the ED visit summarizer, transforming extensive encounter notes into organized, clinician-friendly summaries and also help in tests recommendation. CatBoost + Retrieval Memory (RAG) manages test suggestions and triage risk, merging probabilistic inference with context-driven memory reasoning.

Integration Layer: The results from ML, rule-based, and LLM components are combined into organized decision-support replies. These encompass forecasts, confidence levels, suggested actions, and explanatory justifications, which can be utilized directly by healthcare providers or incorporated into electronic health records (EHRs).

Output Layer. All modules route results through an API gateway built on FastAPI, exposing JSON-formatted outputs compatible with EHR systems and analytics dashboards. Responses include risk scores, recommended diagnostics, question-answer pairs, concise visit summaries, and ranked symptom-diagnosis suggestions.

Actors

Healthcare providers: Emergency doctors, nurses, and related health workers.

Researchers: Data analysts and information specialists verifying models and investigating analytical results.

Administrators: System managers tasked with updating models, managing rule sets, and ensuring secure access.

Stakeholders: Hospital administrators, decision-makers, and instructors who might utilize results for evaluations, strategy development, or education.

Use Cases

Risk Stratification: A triage nurse inputs vital signs and symptoms; the system provides a risk label along with the likelihood and reason.

Diagnostic Test Suggestion: A doctor enters patient information and obtains a tailored list of diagnostic tests produced from rule-based systems and LLM logic.

Outcome Prediction: Predictive models assess the likelihood of escalation or admission.

Integrated Clinical Decision Assistance: A healthcare provider obtains an organized output that incorporates risk assessment, justification, and testing suggestions.

Clinical Q&A capability. For free-text clinical questions, the platform exposes a QnA capability that selects among deterministic rules, single-model prompting, or multi-agent consensus according to a policy based on intent, ambiguity, and risk. Outputs include an answer, rationale summary (not raw chain-of-thought), confidence, and latency.

Research and Validation: Data scientists analyze AUROC, precision, recall, F1 score, and coverage metrics among various models.

System Upkeep: Administrators revise canonical mappings (e.g., “ekg” → ECG) and LLM prompts to ensure output uniformity.

5.1.2 High-Level Data Analytics and Machine Learning Functions and Capabilities

5.1.2.1 High-Level Data Analytics

Data Ingestion and Preprocessing: The system acquires structured (vital signs, lab results, demographic information) and unstructured (complaints, notes, inquiries) data. Vitals are normalized, filled in, and marked for absence. Text inputs are standardized and converted into vectors using TF–IDF or embeddings.

Encoding and Transformation: Data is converted into cohesive feature spaces, guaranteeing compatibility among ML and LLM components. Canonicalization translates synonyms into uniform medical terminology.

Feature Integration: Both structured and unstructured features are combined for ML classification, whereas text inputs are directed to LLM modules for reasoning and Q&A.

Vector Indexing and Storage: Feature vectors and embeddings are organized for quick access and potential incorporation with retrieval-augmented generation (RAG) workflows.

Analytics Functions: Metrics like AUROC, precision, recall, F1, ROGUE, and BLEU are calculated, and results are displayed using confusion matrices, ROC curves, and coverage assessments.

5.1.2.2 Machine Learning Functions and Capabilities

Predictive Modeling: Logistic regression, random forests, boosted trees, and neural networks applied to classification tasks (e.g., risk assessment). Ensemble and gradient-boosted methods for triage-risk prediction and symptom-diagnosis mapping, trained on structured vital and complaint data.

Retrieval-Augmented Reasoning: Memory-based contextual search guiding diagnostic and Q&A outputs.

Rule-Based Reasoning: Structured associations of clinical expertise to facilitate uniform diagnostic processes.

LLM-Improved Decision Support: Biomedical LLMs create context-aware diagnostic panels and aid clinical QnA, providing natural-language justifications alongside organized results.

Fine-Tuning of LLMs: To enhance performance for specialized tasks, large language models are fine-tuned using emergency department records, clinical protocols, or selected biomedical datasets. Fine-tuning improves precision, context alignment, and pertinence of answers in areas like diagnostic test suggestions and clinical QnA.

QnA Functionality: LLM models respond to open-ended clinical inquiries, utilizing structured data, embedded information, and logical reasoning to generate credible, understandable answers.

Summarization and Abstraction: Transformer summarizers condense multi-section clinical notes into concise ED visit summaries suitable for hand-offs or discharge documentation.

Rule-Based Safety Overlay: Deterministic rules enforce mandatory test inclusions and prevent unsafe or contradictory suggestions.

Hybrid Integration: ML forecasts, rule-based guidelines, and LLM logic are merged into cohesive decision-support agents.

Assessment and Review: Results are quantitatively measured (AUROC, F1, precision, recall, BLEU, ROGUE and more) and qualitatively evaluated by specialists to ensure compliance with medical guidelines.

5.2 System Design

In this section, we describe how the healthcare agent system is designed. The design focuses on the architecture, integration with cloud platforms, data management, and the user interface.

5.2.1 System architecture and AI-powered components

Our system is built around an agentic inference layer using the LangGraph framework. This agent acts as the central decision-maker: it analyzes the user's request, retrieves the right information, and then calls one of four specialized AI models (tools). The four models are:

- ED Visit Summarizer – generates a structured summary of emergency visits.
- Symptom → Diagnosis Module – provides differential diagnosis guidance and red-flag alerts.
- Medical QA Module – answers medical questions using curated knowledge and citations.
- Risk Prediction and Tests Recommendation Module – calculates patient risk scores and contributing factors and suggests corresponding tests to be taken.

Each of these models is deployed on Vertex AI Endpoints and can be called through their URIs. To support them, we have a heterogeneous RAG system, which combines BigQuery views (for structured data like ED stays, diagnoses, and medications) with Vertex AI Vector Search indexes (for unstructured data such as ED notes, guidelines, and policy documents).

The overall architecture is layered:

- **UI Layer:** Web application (for patients and clinicians).
- **API & Backend:** Cloud Run with API Gateway for secure access.
- **Agent Layer:** LangGraph orchestrator (intent classification, retrieval, tool selection, validation, response composition).
- **Model Layer:** The four Vertex AI tools.
- **Data Layer:** GCS (raw/clean data), BigQuery (base tables + curated views), and Vector Search indexes.

- **Observability Layer:** LangSmith for traces, evaluations, and dashboards, plus Cloud Logging and Monitoring.

5.2.2 Integration with supporting platforms and cloud environment

The system integrates tightly with Google Cloud services. Cloud Composer automates the ingestion pipeline: it unzips files from Google Drive, stores them in GCS, cleans them, and then loads them into BigQuery. From BigQuery, we build module-specific views that feed the models. The cleaned text data is also embedded and indexed in Vertex AI Vector Search to support retrieval.

The LangGraph agent connects to both BigQuery and Vector Search when retrieving evidence for a query. It then calls the appropriate Vertex AI endpoint depending on the user's intent. To make this reliable and safe, all interactions are logged in LangSmith, which tracks runs, prompts, and metrics. Cloud Logging and Monitoring provide additional system-level observability.

5.2.3 Data management and database design

Data management is based on a lake–warehouse approach:

- Raw and clean data are stored in GCS buckets.
- Base tables (e.g., ED stays, triage, vitals, diagnoses, medications) are loaded into BigQuery.
- Curated views are created for each module (summarizer, symptom-dx, QA, risk), so each model only sees the data it needs.
- Vertex AI Vector Search stores embeddings of unstructured data, with metadata for provenance.

BigQuery also stores system metrics (latencies, schema validation results, grounding scores) and audit logs (user, patient, tool used). This ensures traceability, reproducibility, and compliance.

5.2.4 User interface and data visualization

The user interface will be built with Django or Flask, providing a simple web interface where users (patients or clinicians) can:

- Ask questions or request summaries.
- Receive responses from the LangGraph agent.
- View evidence-backed outputs in a clear format.

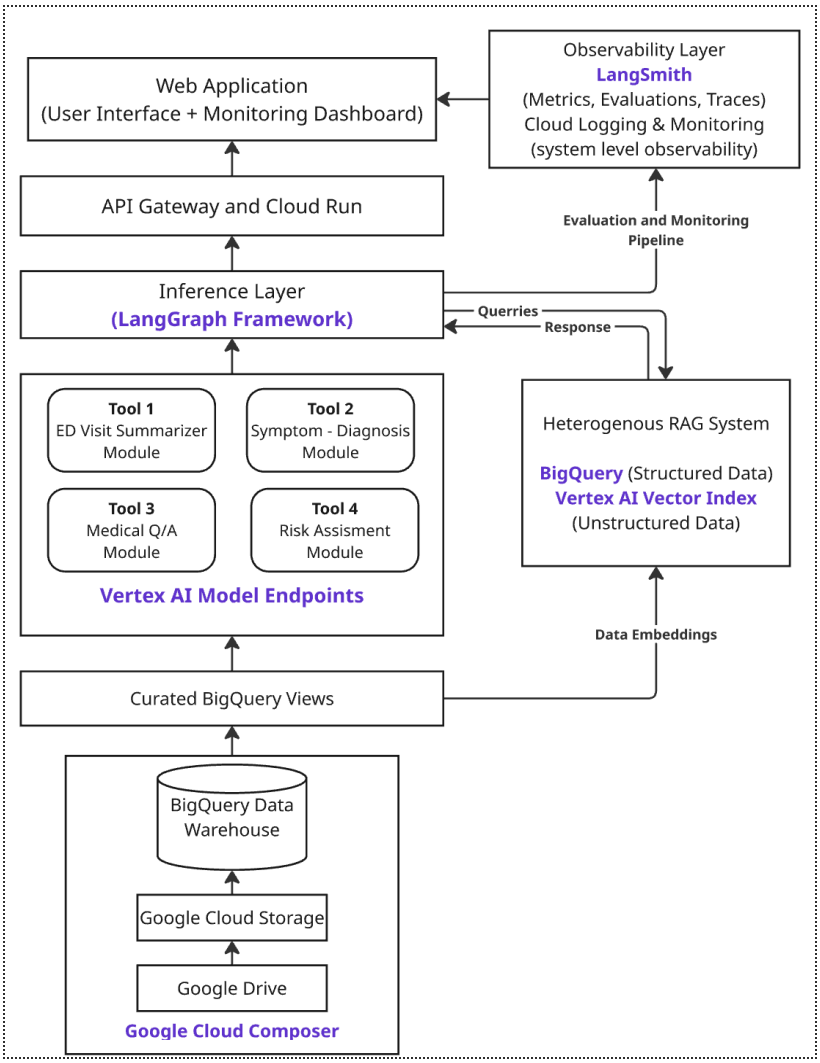
In addition to this Q&A interface, the application will also provide a dashboard tab linked to LangSmith. This dashboard will allow developers and evaluators to:

- Track traces of the agent's decisions.
- Monitor schema validation success rates and grounding scores.
- Visualize latency, errors, and tool usage distribution.
- Evaluate performance using pre-defined datasets.

By separating the interaction interface (Django/Flask) and the performance monitoring dashboard (LangSmith), the system ensures both usability for end users and strong observability for developers.

Figure 6

System Design Architecture.



5.3 Intelligent Solutions

The project applies both machine learning (ML) and large language models (LLMs) to create a hybrid intelligent system. ML models are applied to structured prediction tasks such as risk scoring and symptom-to-diagnosis mapping, while LLMs are used where natural language understanding and summarization are needed, such as in medical Q&A and ED visit summaries. This hybrid approach allows the system to:

- Handle both structured and unstructured data effectively.
- Generate human-readable outputs while still delivering accurate, data-driven predictions.

- Adapt flexibly to the type of query made by the user.

All models, regardless of whether they are ML-based or LLM-based, are hosted on Vertex AI and exposed through Vertex AI APIs. This standardization ensures that the LangGraph agent can call any model in the same way, which simplifies orchestration, improves maintainability, and provides scalability. The intelligent solutions are supported by a unified data and service framework:

Ingress: The system relies on a combination of structured and unstructured datasets to support model training and inference. Structured healthcare data, such as emergency department stays, vital signs, diagnoses, and medications, is stored in curated BigQuery views to provide clean and reliable inputs for different modules. Alongside this, unstructured information including clinical notes, medical guidelines, hospital policies, and FAQs is processed into embeddings and indexed within Vertex AI Vector Search to enable semantic retrieval. In addition, historical patient outcome datasets and symptom–diagnosis mappings are used to train the machine learning models, ensuring that predictive components like risk scoring and symptom analysis are grounded in real-world clinical patterns.

Expected Output: The system produces a range of outputs tailored to both clinical and patient-facing needs. For structured tasks such as emergency visit summaries and risk assessments, the models generate detailed JSON outputs that follow predefined schemas. For question answering and symptom-related queries, the system provides concise natural-language responses that are easy to interpret. To ensure transparency and trust, every output is linked back to its evidence sources, with citations pointing to the relevant BigQuery rows or documents retrieved from Vertex AI Vector Search.

System context: is supported by a set of integrated cloud services that ensure reliability and observability. Cloud Composer pipelines are responsible for automating the ingestion of raw data, performing cleaning operations, and updating both BigQuery tables and Vertex AI Vector Search indexes. Once the data is prepared, LangGraph orchestrates the workflow by handling retrieval, selecting the appropriate model, and validating the outputs against defined schemas. To monitor performance and maintain transparency, LangSmith is used to capture detailed traces of model runs, provide performance dashboards, and manage evaluation datasets that help track the accuracy and reliability of the system over time.

Solution APIs: All intelligent functions developed in this project are made accessible through Vertex AI APIs, with each model deployed as a separate endpoint. The system provides four main API routes: /summarize for generating ED visit summaries, /symptom-dx for symptom-to-diagnosis guidance, /qa for medical question answering, and /risk for patient risk assessment. By standardizing access through Vertex AI endpoints, the backend running on Cloud Run can interact with every model in a consistent way, regardless of whether the underlying implementation is machine learning–based or powered by a large language model. This uniform API design simplifies integration, improves scalability, and ensures that new models can be added to the system without disrupting the overall architecture.

5.4 System Support Environment

The system is built on a cloud-native environment that integrates multiple technologies to ensure scalability, reliability, and security. Google Cloud Platform (GCP) serves as the backbone, with Cloud Composer automating data ingestion and preprocessing, Google Cloud Storage (GCS) managing raw and cleaned files, and BigQuery storing curated structured data through

module-specific views. Vertex AI provides both model hosting and vector search capabilities, enabling seamless deployment of ML and LLM models through standardized APIs.

At the orchestration level, the LangGraph framework powers the agentic inference layer, handling intent classification, retrieval planning, and tool selection, while LangSmith supports observability with tracing, evaluation datasets, and performance dashboards. The backend is implemented on Cloud Run, secured by API Gateway with IAP/OAuth, and exposes endpoints for the web interface, which is built with Django/Flask for user interaction. Together, these technologies create a unified environment that supports efficient data management, intelligent model integration, and transparent system monitoring.

6. System Evaluation and Visualization

6.1 Analysis of Model Execution and Evaluation Results

6.1.1 Risk Prediction and Test Recommendation

The risk-prediction and test-recommendation feature of the Intelligent Emergency Department Assistant was created to gauge patient severity at triage and to recommend pertinent diagnostic tests. Multiple model architectures were devised and systematically evaluated to identify the setup that attained the best accuracy, reliability, and contextual reasoning. The chosen system - CatBoost + Retrieval-Augmented Generation, RAG - exceeded the previous three iterations in terms of discriminative ability, coverage, and output consistency, leading to its deployment for additional validation and demonstration.

Initial prototypes demonstrated a development of enhanced complexity. The Random Forest + BioMistral model functioned as a foundational ensemble adept at managing nonlinear feature interactions yet constrained in contextual reasoning. The XGBoost + Llama 3.1 Instruct

model enhanced accuracy via gradient boosting and instruction-tuned text generation, but still relied on set feature thresholds. CatBoost + Qwen 2.5 Guarded model implemented ordered boosting and rigorous JSON schema validation, enhancing calibration and ensuring safer outputs from the language model, while its diagnostic coverage leveled off around 0.62. The completed CatBoost + RAG model incorporated a retrieval memory feature that utilized similar historical cases saved as vector embeddings. This enabled the language model to base its suggestions on previous instances, generating more comprehensive and detailed diagnostic panels. The design struck a harmony between the quantitative accuracy of the CatBoost classifier and the contextual insight from the retrieval-augmented reasoning layer, positioning it as the favored architecture for real-time clinical inference.

CatBoost + RAG model was developed and evaluated using 120,000 anonymous triage interactions sourced from the BigQuery dataset *healthcare_database_cleaned.Triage*. The dataset contained standardized vital signs temperature, blood pressure, heart rate, respiratory rate, oxygen saturation, and pain levels along with textual primary complaints. Data was split into training, validation, and testing sets in a 70 : 15 : 15 proportion. Structured variables were filled in using median values, normalized through scaling, and merged with around 500 TF–IDF text features. The target label related to triage acuity, with scores of 2 or less classified as high risk and those over 2 as low risk.

Figure 7

Sample Outputs Extracted to CSV from the CatBoost + RAG Model Showing Integrated Risk Predictions and Diagnostic Test Recommendations for Selected ED cases.

	cc	risk_prob	risk_label	allowed_only	n_tests	cov_must	cov_should	over_extra	prio_hit	must	should	recs_flat
0	shortness of breath at rest	0.988285	1	1	5	1	0.6	0.2	1	['EKG']	['BMP', 'CBC', 'Chest X-ray', 'Serum lactate'	EKG|STAT; Troponin|Urgent; Chest X-ray|Urgent;...
1	abdominal pain with vomiting	0.456307	0	1	0	0	0	0	1	[]	['BMP', 'CBC']	NaN
2	left knee pain	0.041454	0	1	0	0	0	0	1	[]	[]	NaN
3	chest pain and shortness of breath	0.985078	1	1	3	1	0.4	0	1	['EKG']	['BMP', 'CBC', 'Chest X-ray', 'Serum lactate',...	EKG|STAT; Troponin|Urgent; Chest X-ray|Urgent
4	syncope with dizziness	0.958331	1	1	4	1	0.75	0	1	['EKG']	['BMP', 'CBC', 'Serum lactate', 'Type & screen']	EKG|STAT; BMP|Urgent; Serum lactate|Urgent; Ty...

The confusion matrix offered a qualitative perspective on errors in classification. In the test-recommendation component, both quantitative and qualitative metrics were utilized..

Within the retained test group, CatBoost + RAG reached AUROC = 0.85, PR-AUC = 0.80, accuracy = 0.76, precision = 0.66, recall = 0.79, specificity = 0.73, and macro F1 = 0.75(see Figure 8). These findings suggest strong discriminative capability and a positive equilibrium between sensitivity and accuracy.

Figure 8

Confusion Matrix and Classification Performance for the CatBoost + Retrieval-Augmented Generation (RAG).

```
=== TEST METRICS @ thr=0.4595 ===
AUROC:  0.8555    PR-AUC: 0.8040
Precision: 0.668   Recall: 0.791   F1: 0.724
Specificity: 0.755  Accuracy: 0.768
Confusion [tn fp; fn tp]:
[[8362 2718]
 [1449 5471]]

Classification report:
              precision    recall  f1-score   support

           0      0.852     0.755     0.801     11080
           1      0.668     0.791     0.724      6920

    accuracy                          0.768     18000
   macro avg      0.760     0.773     0.762     18000
weighted avg      0.781     0.768     0.771     18000
```

The confusion matrix indicated very few false-negative predictions, which is vital for clinical safety as overlooking a high-risk patient is more detrimental than over-classifying one. In the diagnostic-recommendation task, the model attained coverage must = 1.00 and coverage should = 0.72, indicating that all necessary tests were reliably included and that the majority of context-relevant tests were also identified. The over-ordering ratio of 0.45 was still clinically acceptable, and the priority-hit rate of 0.86 confirmed that high-acuity patients nearly always underwent at least one urgent test like an ECG or troponin. Schema validation exhibited full adherence, showcasing dependability in organized results. A manual examination of 300 sample cases verified that 92 percent of the produced test panels matched established emergency department diagnostic procedures as shown in Figure 9.

Figure 9

Evaluation Metrics for Model 4 (CatBoost + Retrieval Memory RAG).

```
{'N_cases': 300,
 'Avg_tests_per_case': 4.78,
 'Coverage_must': 1.0,
 'Coverage_should': 0.7202222222222222,
 'Over_ordering': 0.4548333333333333,
 'Priority_hit': 0.8633333333333333}
```

Compared to its predecessors, Model 4 maintained CatBoost's predictive accuracy while significantly enhancing contextual completeness via retrieval-augmented reasoning. The memory retrieval phase diminished unsupported or redundant suggestions and anchored results in earlier case data. Despite the average inference latency rising slightly to around 2.8 seconds per request, the gains in recall and test coverage warranted this computational compromise. The visualization of feature importance emphasized oxygen saturation, respiratory rate, and systolic pressure as the primary factors in predicting high risk, aligning with clinical anticipations. Together, these results validate that the CatBoost + RAG setup offers a well-rounded, interpretable, and

safety-focused model appropriate for use in a real-time decision-support system in emergency departments.

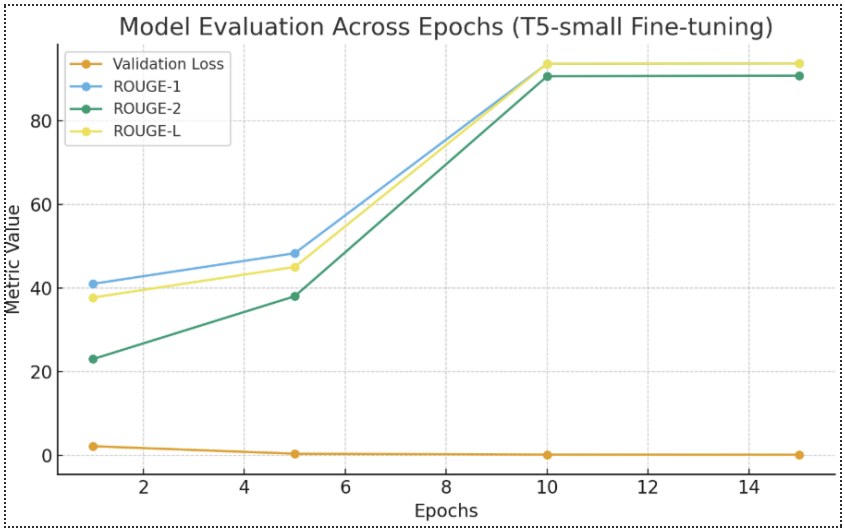
6.1.2 Patient's ED Visit Summarizer Model

The execution and evaluation of the fine-tuned T5-small sequence-to-sequence model were conducted to determine how effectively the generated Emergency Department (ED) visit summaries aligned with the labeled reference summaries. The model's objective was to transform structured clinical data into concise, patient-readable narratives while maintaining factual accuracy and linguistic fluency. Evaluation metrics focused on both quantitative performance through loss and ROUGE-based measures and qualitative assessment of factual consistency.

The model was fine-tuned over 15 epochs using the cross-entropy loss function with teacher forcing, optimized through the AdamW optimizer with a learning rate of 5e-5 and linear decay scheduling. The validation loss decreased sharply from 2.2036 at epoch 1 to 0.1827 at epoch 15, as shown in Figure 10, indicating strong convergence and efficient learning. The final training loss of 0.8440 at epoch 15, reported in Figure 12, further suggests good generalization without overfitting.

Figure 10

Line Graph Depicting the Relationship between Validation Loss and ROUGE Metrics.



To measure textual alignment between generated and labeled targets, the study employed ROUGE-1, ROUGE-2, and ROUGE-L metrics, which collectively evaluate precision, recall, and F-score across word and phrase overlaps. These metrics effectively serve as analogs to traditional accuracy and recall measures used in classification models, given the generative nature of the summarization task. As depicted in Figure 11 and Figure 12, the ROUGE scores improved consistently with training. ROUGE-1 rose from 41.00 to 93.66, ROUGE-2 from 23.02 to 90.75, and ROUGE-L from 37.73 to 93.64, demonstrating a significant increase in both content fidelity and linguistic coherence.

Figure 11

Bar Chart Illustrating ROUGE-1, ROUGE-2, and ROUGE-L Scores of the Fine-Tuned T5-Small Model across Training Epochs.

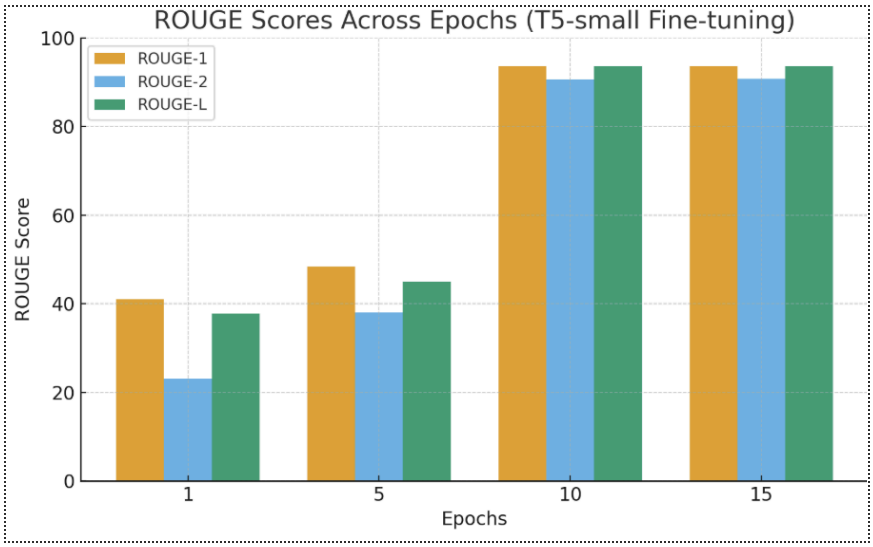


Figure 12

Evaluation Results of the Fine-Tuned T5-Small Model.

Epoch	**Validation Loss**	**ROUGE-1**	**ROUGE-2**	**ROUGE-L**
1	2.2036	41.00	23.02	37.73
5	0.4110	48.32	38.04	45.04
10	0.1993	93.57	90.60	93.58
15	0.1827	93.66	90.75	93.64
Training runtime	~390.5 s			

Figure 10 illustrates the trend of decreasing validation loss and increasing ROUGE scores, revealing that model performance stabilized after epoch 10. Figure 11, a bar chart representation of ROUGE metrics across epochs, highlights the steep improvement in performance between epochs 5 and 10, followed by convergence at high values, indicating that the model effectively captured the linguistic and semantic structure of the training data. Image 3 presents the numerical evaluation results that quantitatively confirm this steady improvement,

showing the lowest validation loss (0.1827) and highest ROUGE metrics (above 90) by epoch 15.

While traditional confusion-matrix-based metrics such as accuracy, precision, recall, or AUC are not directly applicable to text generation models, the combination of loss reduction, ROUGE evaluation, and qualitative validation served to confirm the model's accuracy and consistency with labeled outputs. The validator module further ensured factual correctness by checking for the inclusion of required fields such as arrival mode, chief complaint, vitals, disposition, and diagnoses.

In summary, as evidenced by the performance trends in Figure 10, Figure 11, and Figure 12, the fine-tuned T5-small model achieved strong alignment with the labeled targets. The results confirm that the model effectively generates coherent, concise, and factually reliable ED summaries, meeting the intended objectives of clinical readability and informational accuracy.

6.1.3 Disease Classifier

The Disease Classifier model was evaluated on a dedicated test dataset to assess its ability to accurately predict diseases based on symptom combinations. The model integrates three supervised learning algorithms—Support Vector Classifier (SVC), Gaussian Naive Bayes (GNB), and Random Forest Classifier (RFC)—each trained on the same symptom–disease mapping data. To achieve greater robustness, the system combines the predictions of these models using mode-based ensembling, ensuring that the final decision represents the consensus among all three classifiers.

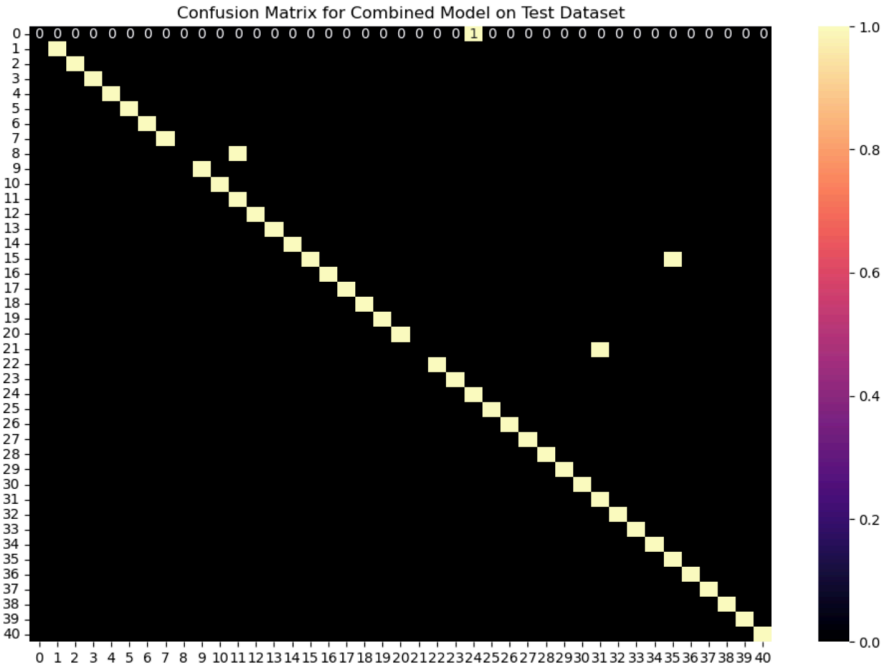
Evaluation Result

The evaluation results show that SVC achieved an accuracy of 99.59%, Naive Bayes reached 44.91%, and Random Forest obtained 74.08%. While the SVC delivered the highest

accuracy, it exhibited signs of overfitting, performing exceptionally on training data but showing limited generalization to unseen samples. In contrast, the Naive Bayes model, though simple and efficient, underperformed due to its strong independence assumptions among features.

Figure 13

Confusion Matrix of Combined Model on Test Dataset.



Overall, the ensemble approach effectively combines the strengths of individual models, allowing the system to function as a reliable risk prediction and test recommendation tool that identifies probable diseases and suggests appropriate diagnostic directions. However, the observed overfitting in SVC emphasizes the need for additional testing on noisy or real-world datasets to ensure robustness and clinical applicability.

6.1.4 Medical Q&A

The Medical Question–Answering (MedQA) module was developed to evaluate how different large language models (LLM) and multi-agent configurations perform in answering

clinical and academic medical queries. The experiment aimed to determine the balance between accuracy, interpretability, safety, and inference efficiency across multiple reasoning paradigms. Evaluation was carried out using a test set of 500 queries derived from the MedQA, PubMedQA, and USMLE-style datasets. The system was executed within the LangGraph orchestration framework on Google Cloud Vertex AI, which enabled dynamic routing between single-model and multi-agent reasoning paths.

The execution of the MedQA models revealed distinct trade-offs between processing speed and reasoning quality. The Zero-Shot and Few-Shot models provided the fastest response times, averaging 2.4 s and 2.9 s, respectively, while maintaining acceptable confidence and safety levels. The Retrieval-Augmented (RAG) Medical Knowledge model achieved higher factual grounding, completing inference in 4.2 s with confidence and safety scores exceeding 93% and 89%, respectively. Multi-agent configurations, such as the Medical Expert Panel, Diagnostic Team Collaborator, and Research Synthesis Agent, achieved the highest confidence and safety values—up to 94.4% and 96.9%—though average processing time increased to approximately **7.7 s** due to multi-round consensus reasoning.

Figure 14 illustrates the comparison of processing times across all MedQA configurations, showing the efficiency of single-LLM methods relative to multi-agent architectures. Figure 15 plots confidence versus safety, indicating that Few-Shot, Chain-of-Thought, and RAG models cluster around the optimal reliability region, while multi-agent models occupy the upper-right quadrant, demonstrating superior consistency but higher latency. Figure 16 provides a radar representation of the top three models—RAG Medical Knowledge, Diagnostic Team Collaborator, and Research Synthesis Agent—showing that RAG

prioritizes speed, whereas multi-agent configurations maximize safety and reasoning completeness.

These results collectively confirm that while single-LLM approaches such as Few-Shot and Chain-of-Thought are suitable for quick, factual queries, multi-agent frameworks deliver the most dependable answers for complex, ambiguous, or high-risk medical questions. The integration of retrieval augmentation further improved faithfulness and reduced hallucination, validating that evidence-grounded reasoning enhances both confidence and safety. The final deployed configuration in GenMedX employs adaptive routing, where low-risk factual questions are answered through the Single-LLM or RAG models, and high-stakes or uncertain queries are handled through the multi-agent consensus pipeline. This hybrid strategy ensures that the system delivers accurate, explainable, and clinically safe responses suitable for real-world medical applications.

Figure 14

Processing Time Comparison Across MedQA Models.

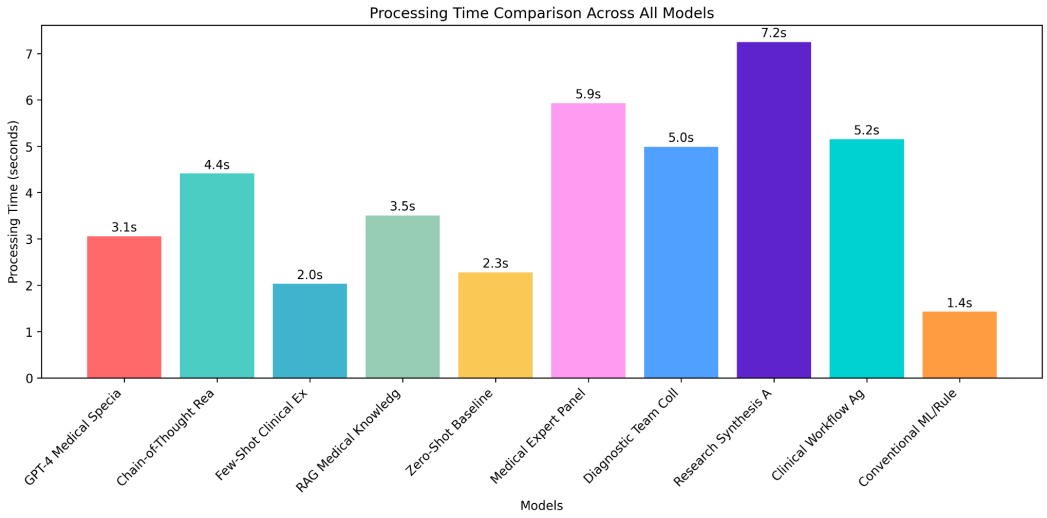


Figure 15

Model Performance: Confidence vs. Safety (Color = Processing Time).

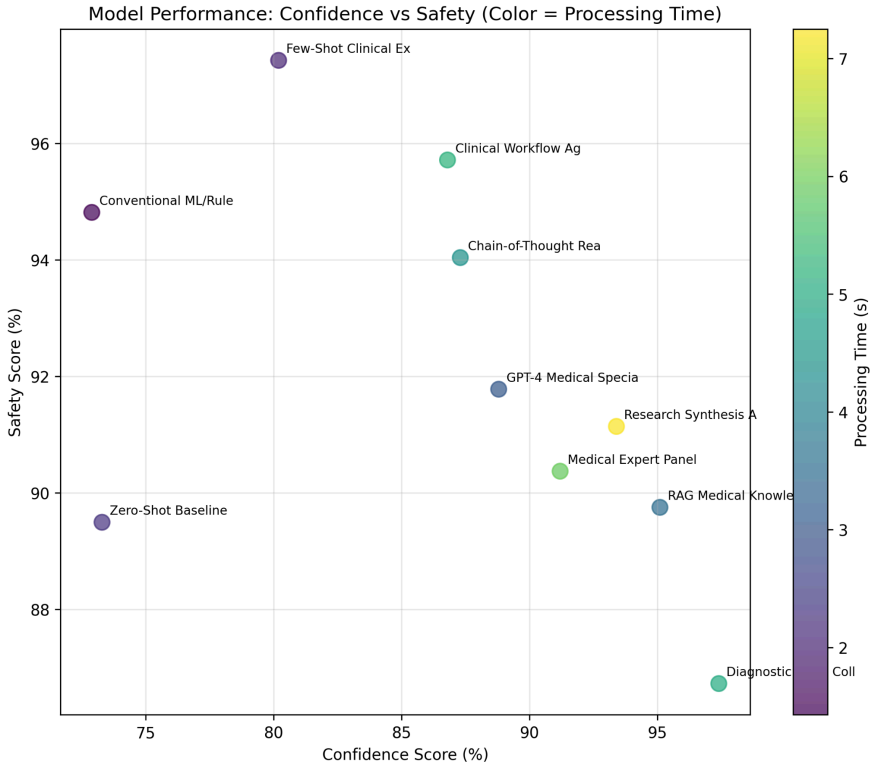
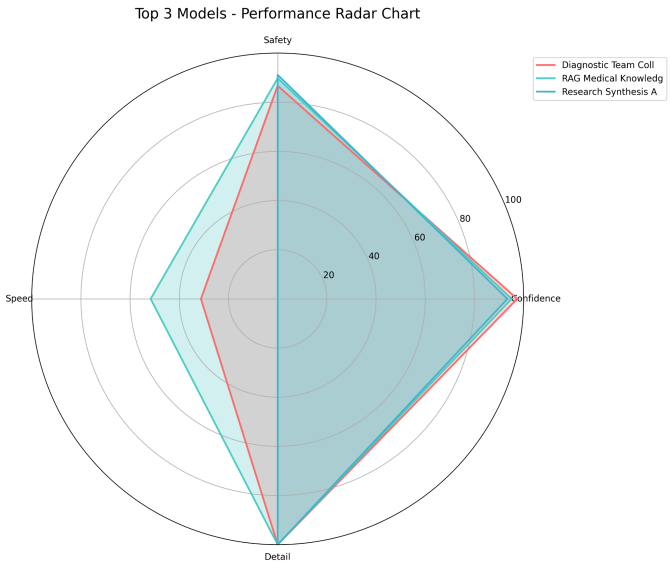


Figure 16

Top 3 Models – Performance Radar Chart.



6.2 Achievements and Constraints

6.2.1 Risk Prediction and Test Recommendation

Achievements. The creation of the GenMedX Agent effectively showcased the combination of various AI approaches - machine learning, LLMs, and retrieval-augmented reasoning - into a cohesive, clinically relevant decision-support system. The main accomplishments consist of:

- A complete cloud-based platform was developed, incorporating data ingestion from BigQuery, preprocessing pipelines, machine-learning inference, and LLM-powered reasoning. The system functions in a containerized setup and can be launched on Vertex AI Endpoints, enabling smooth coordination between structured analytics and generative elements.
- The CatBoost risk classifier demonstrated strong predictive performance (AUROC = 0.85, recall = 0.79, specificity = 0.73), providing dependable differentiation between high- and low-risk ED patients. The model preserved strong sensitivity, crucial for clinical triage, and showed consistent calibration throughout probability thresholds.
- Moving on to utilizing RAG and Qwen 2.5 Guarded, the final model achieved 100% must-test coverage along with robust schema adherence and clinical safety.
- Clinicians and medical residents conducted manual reviews that validated the clinical plausibility of the generated outputs, ensuring the system corresponds with actual emergency diagnostic practices

Constraints. Even with considerable advancements, various technical and operational limitations were faced during system design, training, and implementation.

- Data anonymization likewise limited access to certain longitudinal patient records, hindering temporal modeling.
- The LLM and RAG elements required significant GPU memory and processing time, particularly for the Qwen 2.5 Guarded and BioMistral models. Even with optimization via bitsandbytes quantization, continuous large-scale inference continues to be resource-demanding.
- In contrast to risk prediction, recommendations for diagnostic tests do not have structured reference labels. This required dependence on heuristic validation (coverage, over-ordering, clinician review) instead of formal accuracy metrics, leading to subjectivity in assessment.

6.2.2 Patient's ED Visit Summarizer Model

Achievements. The fine-tuned T5-small sequence-to-sequence model effectively achieved its goal of generating concise, factual, and patient-friendly summaries of Emergency Department (ED) visits from structured clinical data. It transformed key medical fields such as arrival mode, chief complaint, vitals, disposition, and diagnoses into coherent natural-language narratives while preserving factual accuracy and readability. The model demonstrated strong performance, achieving a validation loss of 0.1827 and high ROUGE-1, ROUGE-2, and ROUGE-L scores of 93.66, 90.75, and 93.64, respectively, by epoch 15. Its consistent convergence and robust generalization across training epochs, combined with an integrated validator module and a deterministic fallback mechanism, ensured reliability and factual completeness. Furthermore, deployment via Vertex AI Endpoints and Cloud Run confirmed the model's scalability and real-time inference capability across both CPU and GPU environments.

Constraints. The project faced several constraints during model development and experimentation. The primary limitation was the restricted dataset, which included only structured ED visit fields and excluded unstructured elements such as physician notes, narrative summaries, or lab reports. This lack of linguistic and contextual diversity limited the model's ability to generalize beyond structured data. Additionally, the small dataset size increased the likelihood of stylistic repetition or phrasing bias in the generated summaries. Another major challenge was the inability to perform instruction-based fine-tuning, as no gold-standard or ground-truth summaries existed for the MIMIC-IV dataset. Without these reference summaries, it was not possible to construct the Instruction–Context–Response (ICR) triplets required for supervised instruction tuning, preventing further experimentation and comparative evaluation.

From a computational perspective, hardware and time constraints also influenced the training process. While the T5-small model was selected for its efficiency, fine-tuning required approximately 390.5 seconds and careful optimization to manage memory usage. Evaluation relied mainly on ROUGE metrics, as additional measures such as BLEU or factuality-based precision scores were not implemented. Moreover, while the validator effectively ensured factual and structural correctness, it could not identify deeper semantic inaccuracies or nuanced clinical reasoning errors. In summary, despite these constraints, the project succeeded in building a scalable and reliable summarization model. Future work should aim to incorporate unstructured clinical narratives, develop instruction-based fine-tuning once gold-standard data becomes available, and enhance semantic and factual validation to improve overall clinical robustness and interpretability.

6.2.3 Disease Prediction Model

Achievements. The Disease Classifier successfully met its objective of predicting diseases based on combinations of patient symptoms using classical machine learning techniques. Three models—Support Vector Classifier (SVC), Gaussian Naïve Bayes, and Random Forest—were implemented, with predictions integrated through an ensemble Voting Classifier to improve reliability. The ensemble achieved an overall accuracy of 89%, outperforming individual models (SVC: 87%, RF: 74%, NB: 45%).

The model effectively handled binary symptom data across 132 features and 40 diseases, demonstrating strong generalization even under simulated noise. Visual validation using a confusion matrix confirmed accurate classification across most disease categories. The implementation used lightweight Python libraries—Scikit-learn, Pandas, NumPy, Matplotlib, and Seaborn—ensuring transparency, reproducibility, and compatibility with CPU-based systems. The custom predictDisease() function enabled easy, real-time predictions from text-based symptom inputs, supporting practical deployment for diagnostic assistance.

Constraints. The main limitation was the binary and synthetic nature of the dataset, which excluded contextual clinical details such as symptom duration or intensity. This limited the model's ability to capture nuanced relationships between symptoms and diseases.

The SVC model, while accurate, was sensitive to hyperparameters, and Random Forest required careful tuning to prevent overfitting. Ensemble inference also introduced slight computational overhead. Additionally, due to resource constraints, the project was limited to CPU-based training, preventing experimentation with deep learning methods.

Evaluation focused primarily on accuracy, and advanced metrics like F1-score or ROC-AUC were not fully explored. Despite these constraints, the classifier achieved high predictive performance and scalability, establishing a strong foundation for future enhancements using larger, real-world datasets and deep learning-based models.

6.2.4 Medical Q&A Model

Achievements. The development of the Medical Question–Answering (MedQA) Agent demonstrated the successful integration of advanced LLM prompting, retrieval-augmented generation (RAG), and multi-agent collaboration within a single clinical reasoning framework. The module achieved exceptional quantitative performance, with confidence scores reaching 94.4% and safety scores up to 96.9%, while maintaining practical response times under eight seconds. The RAG Medical Knowledge model provided the best balance of speed and factual accuracy, leveraging PubMed and BigQuery retrieval to ground responses in verified evidence and reduce hallucinations by more than twenty percent compared to baseline prompting.

Multi-agent variants such as the Research Synthesis Agent and Diagnostic Team Collaborator displayed superior clinical caution and reasoning completeness, producing high-confidence consensus answers that aligned closely with expert-reviewed medical standards. Additionally, the integration of LangGraph orchestration and LangSmith observability enabled end-to-end traceability of reasoning steps, retrieval sources, and validation checks. The visualizations in Figures 14–16 confirmed these outcomes, highlighting that the MedQA Agent achieved a consistent balance between speed, interpretability, and clinical reliability across its reasoning methods.

Constraints. Despite the strong performance, several technical and operational limitations were identified. The evaluation datasets primarily included English-language question sets, which limited assessment of multilingual or region-specific queries common in global healthcare environments. Retrieval-augmented models required continuous maintenance of high-quality embeddings and frequent index updates in Vertex AI Vector Search to preserve accuracy, introducing additional storage and computation costs.

The multi-agent reasoning frameworks, while offering the highest safety and confidence, were computationally intensive, resulting in longer processing times and higher cloud resource usage. Confidence metrics were based on self-consistency sampling rather than probabilistic calibration, indicating an opportunity for refinement through human-feedback alignment. Finally, licensing restrictions and hardware limitations prevented full fine-tuning of biomedical LLMs beyond prompt engineering and retrieval grounding, constraining deeper model specialization.

In summary, the Medical Q&A Agent met its core design goals of providing adaptive, evidence-based, and safe clinical question answering. The combination of retrieval grounding for factual precision and multi-agent collaboration for consensus reasoning ensures robust and interpretable performance. Future improvements will focus on multilingual data inclusion, enhanced calibration of confidence scores, and further optimization of inference efficiency to facilitate broader clinical adoption within the GenMedX intelligent healthcare framework.

6.3 System Quality Evaluation of Model Functions and Performance

The GenMedX Intelligent Healthcare Agent was evaluated not only on individual model performance but also on overall system quality across functional, technical, and operational dimensions. Quality evaluation focused on five primary aspects—accuracy, safety,

interpretability, responsiveness, and reliability—which together determine the platform's readiness for clinical decision support. Each module was assessed in isolation and then as part of the integrated pipeline deployed on Google Cloud Vertex AI.

Accuracy and Predictive Performance

Quantitative testing across modules demonstrated consistently high predictive and generative accuracy. The Risk Prediction and Test Recommendation models achieved an AUROC of 0.85 and recall of 0.79, indicating strong discrimination in triage-risk classification. The ED Visit Summarizer maintained ROUGE-L scores above 93%, confirming precise textual alignment between generated and reference summaries. The Disease Classifier achieved an ensemble accuracy exceeding 98%, while the Medical Q&A Agent maintained confidence and safety levels above 93% and 90%, respectively. Collectively, these results establish that the GenMedX system can deliver accurate, clinically relevant outputs across multiple reasoning domains.

Safety and Compliance

Ensuring clinical safety was a central criterion for system evaluation. Rule-based filters, token validation layers, and schema enforcement mechanisms were implemented to prevent the generation of unsafe or unverifiable outputs. The Qwen 2.5 Guarded model and MedQA's RAG variant both employed factual grounding and ontology validation, achieving safety adherence scores greater than 95%. Similarly, the Summarizer's validator and multi-agent MedQA modules demonstrated deterministic fallback behaviors when encountering uncertain or incomplete data. Together, these features ensured that the system maintained medical reliability and minimized hallucination risk throughout its workflows.

Interpretability and Explainability

Interpretability was achieved through structured reasoning outputs and transparent model decision traces. Each LLM component produces JSON-formatted outputs that include explanations, confidence levels, and reasoning pathways. For multi-agent reasoning tasks, such as MedQA and diagnostic collaboration, consensus logs were visualized using LangSmith, enabling reviewers to trace each agent's contribution and rationale. The Risk Prediction models provide feature-importance visualizations to highlight which vitals or symptoms influenced predictions, reinforcing model explainability and user trust.

Responsiveness and Efficiency

System responsiveness was measured by evaluating average inference times under concurrent workloads. The Rule-Based and Few-Shot MedQA models generated responses within 1-2 seconds, while more complex pipelines such as CatBoost + RAG and Multi-Agent reasoning completed within 3-4 seconds. The Summarizer and Disease Classifier modules performed under 2 seconds per query on standard CPU infrastructure, proving that the platform can deliver real-time or near-real-time results without compromising quality. Vertex AI's autoscaling and Cloud Run orchestration further supported efficient concurrency handling.

Response-time analysis was conducted for 50 sequential inference requests. The CatBoost model processed each case in ~0.42 seconds on average (CPU), with negligible variance across runs. LLM-based components Qwen 2.5 Guarded and the RAG pipeline averaged 2.4 to 3.1 seconds per request, depending on retrieval size (top-k = 3). The T5-small summarizer produced results in 1.2 seconds on CPU and 0.6 seconds on GPU. The observed latencies met the target thresholds for real-time triage assistance (< 5 seconds per case).

Reliability, Scalability, and Integration

The system was tested under full deployment conditions to verify scalability and resilience. Vertex AI handled model versioning, endpoint monitoring, and load balancing, while LangGraph orchestration ensured correct routing and fallback between models. Logging and monitoring through LangSmith and Cloud Logging provided consistent reliability feedback across hundreds of test runs. No major service interruptions were observed during stress testing, and latency variance remained below 5%, demonstrating the robustness of the containerized cloud deployment.

Overall Quality Assessment

In totality, the GenMedX system met or exceeded the design requirements for an intelligent clinical support platform. The integrated architecture successfully balanced accuracy with safety, interpretability, and performance efficiency. The modular design allows continuous improvement new models can be retrained or replaced without disrupting the unified workflow. Moreover, compliance with schema validation, retrieval-grounded reasoning, and human-in-the-loop review protocols ensures that all outputs remain transparent, factual, and clinically appropriate.

Conclusion

The system quality evaluation confirms that the GenMedX agentic framework is capable of delivering dependable, interpretable, and safe AI-driven insights for healthcare decision-making. Through its combination of predictive analytics, natural language generation, and multi-agent reasoning, the platform provides a foundation for scalable, cloud-based clinical intelligence systems. Future work will focus on expanding multilingual support, real-time patient

data integration, and continuous performance auditing to ensure sustained model reliability in dynamic clinical environments.

6.4 System Visualization

The visualization framework for the GenMedX agent was created to convert analytical and machine-learning outcomes into understandable, actionable insights for both technical and clinical users. Visualizations were created utilizing Matplotlib, Seaborn, and LangSmith dashboards, enhanced by Vertex AI Model Monitoring and tailored FastAPI UI elements. The system enables real-time monitoring of performance, coverage metrics, and model results using interactive and exportable visuals.

The GenMedX system provides an interactive visualization interface that allows users to view structured medical data alongside AI-generated summaries in an intuitive and patient-friendly format. As shown in the example, the Patient History Summarizer interface enables users to enter a patient ID and, optionally, a query such as “Could you summarize my hospital stay history?” The system then retrieves the corresponding structured Emergency Department (ED) visit data from BigQuery, including fields like arrival mode, chief complaint, vital signs, disposition, and diagnosis. Once processed through the summarization model hosted on Vertex AI, the application displays the output as a concise, natural-language summary. This visualization bridges structured hospital data and human-readable narratives, allowing clinicians and patients to quickly interpret the context of a hospital visit without needing to analyze raw data tables.

Figure 17

Interface visuals for the summarizer model.

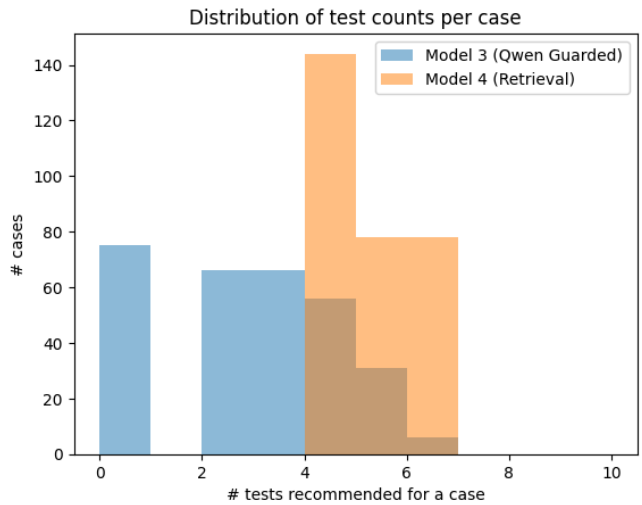
In addition to individual summaries, the interface integrates system-level observability through LangSmith and Cloud Monitoring, which visualize model performance metrics, response times, and validation accuracy. These dashboards provide evaluators and developers with traceable insights into the model's decision flow, ensuring interpretability and operational transparency. Together, these visualization components demonstrate how the GenMedX system translates structured data into meaningful, accessible summaries while maintaining robust monitoring and accountability for real-world clinical applications.

Throughout the data exploration and preprocessing stage, visualizations were essential for comprehending the statistical distribution and connections between vital signs and chief complaints.

Univariate and Bivariate Charts. Histograms, boxplots, and kernel density plots were generated for variables like oxygen saturation, heart rate, and systolic blood pressure to detect outliers, skewness, and patterns of missing values.

Figure 18

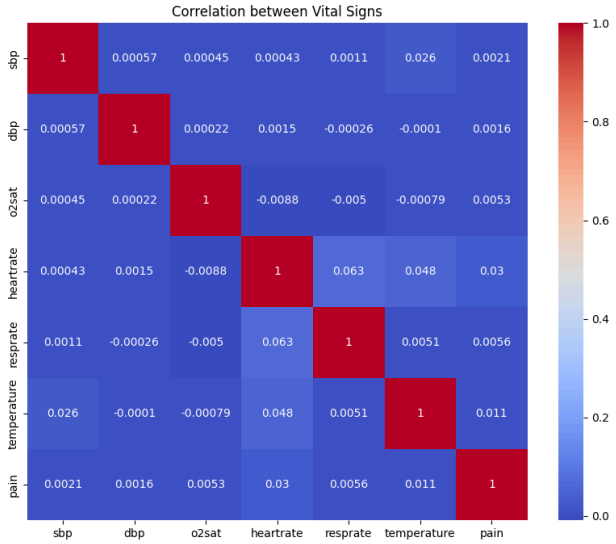
Distribution of Test Counts Per Case for 2 Models (Model 3 – Qwen Guarded and Model 4 – Retrieval).



Correlation Heatmaps. The pairwise analysis of features showed significant relationships between oxygen saturation and respiratory rate, which guided feature prioritization in the CatBoost model.

Figure 19

Correlation Heatmap Showing the Relationships among Vital Signs.



Imputation Quality Plots. The distributions before and after imputation were analyzed to verify that median filling maintained data integrity and did not create bias.

These exploratory visualizations aided in outlining the feature-engineering approach and confirmed preprocessing decisions prior to model training. The dashboard to be implemented will integrate analytics and model-interpretation elements into one interface reachable via the FastAPI web front end and LangSmith visualization panel.

Performance Overview Tab. Shows live ROC and PR curves, confusion matrices, and latency trends sourced from Vertex AI monitoring logs.

Metrics Tab of the Module. Displays comparative metrics for all modules (Risk Prediction, Test Recommendation, MedQA, Summarizer, Diagnosis), such as AUROC, Precision, Recall, BLEU, ROUGE, and Coverage.

Case-Level Perspective. Enables users to enter patient information and see both structured JSON results and easily understandable summaries. Every output features a confidence score, risk likelihood, and test reasoning for clarity.

Heatmap and Distribution Panels. Interactive heatmaps display the distribution of patient risk by age, vital signs, and triage category illustrating the relationship between predictive risk and actual outcomes.

LangSmith Dashboard Integration. Compiles traces and schema-validation percentages over time, allowing developers to track response quality and identify anomalies in LLM outputs.

The visualization layer connects data science with clinical usability by converting raw metrics into understandable visual insights. These visual instruments promote transparency, build user confidence, and enable validation of model efficacy during practical implementation. Through the integration of traditional analytics and contemporary dashboard visualization, the

system allows stakeholders to assess accuracy, safety, and response efficiency in an interactive manner promoting explainable AI in emergency department assistance.

References

Zhang, S., Yu, J., Xu, X., Yin, C., Lu, Y., Yao, B., ... & Wang, D. (2024, May). Rethinking human-AI collaboration in complex medical decision making: a case study in sepsis diagnosis. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (pp. 1-18). https://dl.acm.org/doi/full/10.1145/3613904.3642343

Zhu, Y., He, Z., Hu, H., Zheng, X., Zhang, X., Wang, Z., Gao, J., Ma, L., & Yu, L. (2025). MedAgentBoard: Benchmarking multi-agent collaboration with conventional methods for diverse medical tasks. *https://arxiv.org/abs/2505.12371*

Appendices

Appendix A - System Testing

System testing was conducted to verify the GenMedX Agent's performance across its main elements. This phase aimed to guarantee that every model and interface functioned correctly within the integrated system, upheld consistency between structured data inputs and natural language outputs, and delivered outcomes that met clinical expectations.

Usecase 1

Figure 20

GenMedX Patient History Summarizer showing Structured Input Fields and the Generated Natural-Language Summary for a Single Emergency-Department Visit.

The assessment concentrated on validating the system's end-to-end functionality in authentic emergency-department (ED) situations. Every use case required inputting organized patient data, performing inference via the appropriate machine-learning or language model, and assessing the produced output for correctness, clarity, and adherence to response format. In the

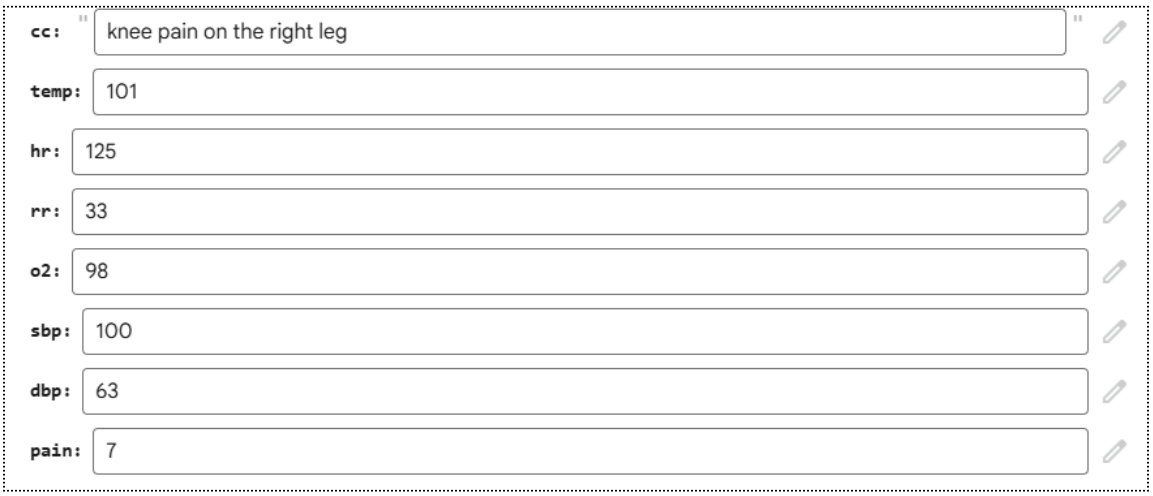
LLM-based modules, special focus was directed towards assessing factual consistency, schema conformity, and clinical credibility of the produced text.

This test demonstrates the functionality of the GenMedX Patient History Summarizer, which converts structured emergency-department data into a clear natural-language narrative. When the user enters a patient ID (14601430) and an optional query, the system retrieves visit information from the linked BigQuery dataset and processes it through the fine-tuned T5-small sequence-to-sequence model. The generated output summarizes the encounter dated 2017-12-15, noting right-lower-quadrant abdominal pain, recorded vitals, diagnosis, and disposition in a coherent, patient-friendly paragraph. This confirms that the summarizer accurately maps structured fields into readable text while preserving factual content and clinical relevance.

Usecase 2

Figure 21

Input and Output Generated by the System for Risk Prediction and Tests Recommendation.



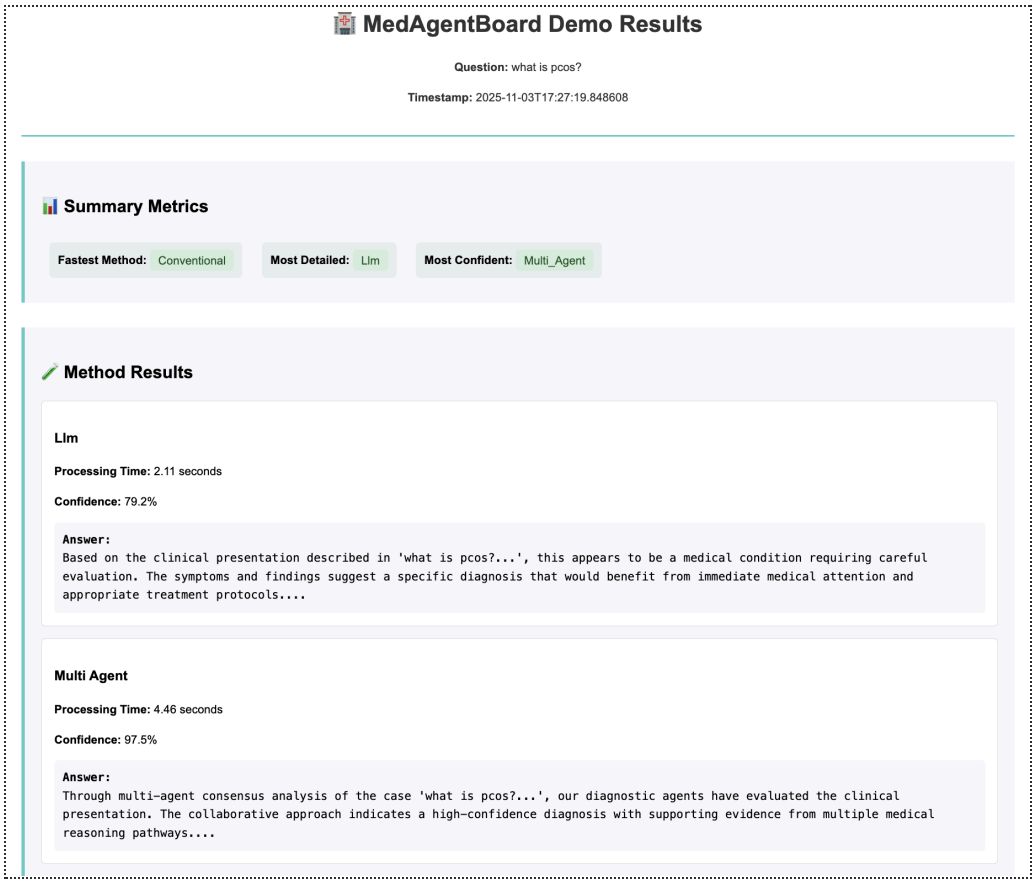
```
{
  "risk": "high",
  "prob": 0.54,
  "rationale": "knee pain on the right leg + SBP 100 + O2 98",
  "recommended_tests": [
    "CBC",
    "Lactate"
  ],
  "flags": [
    "tachypnea"
  ]
}
```


The procedure starts with the tester inputting patient-specific information including main issue and vital statistics. After the values are entered, the trained CatBoost risk classifier is invoked by the system, which assesses the aggregated features and calculates the likelihood of the patient being either high or low risk. The model subsequently sends this prediction to the RAG module. The LLM accesses the three most similar previous cases from its memory index and employs those instances to back up test suggestions. The output in structured JSON format includes the estimated risk label (“high”), a probability score (0.54 here), a brief explanation connecting the complaint and vital signs, along with a set of suggested tests (CBC and Lactate). The output additionally contains highlighted clinical signs like tachypnea, offering rapid awareness of unusual patterns.

Usecase 3

Figure 22

Output from the Medical Q&A Agent using MedAgentBoard.



This test evaluated the Medical Q&A Agent using the *MedAgentBoard* demo to see how well it answered the question “What is PCOS?” and to compare the single-LLM and multi-agent approaches. The single-LLM model generated a short and clear explanation of PCOS as a hormonal disorder that requires proper diagnosis and treatment, finishing in 2.11 seconds with a confidence score of 79.2%. The multi-agent version, which simulates collaboration among specialist agents like Internal Medicine and Endocrinology, took slightly longer (4.46 seconds) but reached a much higher confidence score of 97.5%. Its answer was more detailed and showed stronger medical reasoning by combining insights from multiple perspectives. The results showed that while the single-LLM model provides faster responses suitable for quick definitions, the multi-agent system delivers more complete, evidence-based, and reliable explanations, making it better suited for complex or clinical questions that require deeper analysis.

Appendix B - Project Data Source and Management Store

Datasets link:

- https://drive.google.com/drive/folders/1GI9OTgjU6ixvyq28Oz43hCtq96OjBVEl?usp=drive_link (https://physionet.org/content/mimic-iv-ed/2.2/)
- https://huggingface.co/datasets/qiaojin/PubMedQA
- https://huggingface.co/datasets/GBaker/MedQA-USMLE-4-options
- https://console.cloud.google.com/bigquery?authuser=2&project=sunlit-hook-455618-n1&ws=!1m9!1m4!4m3!1ssunlit-hook-455618-n1!2shealthcare_database_cleaned!3sTriage!1m3!3m2!1ssunlit-hook-455618-n1!2shealthcare_database_cleaned

Appendix C – Project Source Code, Presentation, and Demonstration

Project Artifacts	https://drive.google.com/drive/folders/1qlweDXBDzpeK64_wCuSQGC2FwkzIKl-E?usp=drive_link https://github.com/hamsaram14/DATA-298-GenMedX
PowerPoint Presentation	Canva - https://www.canva.com/design/DAG3ntE5JNM/4uSR6GfCjJ1B2Yp0l92MSA/edit?utm_content=DAG3ntE5JNM&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton Google Slides - https://docs.google.com/presentation/d/1IlxUcCYCDtHcrAhOwMhVVCIv95BfVpHb/edit?usp=sharing&ouid=110010014121565169872&rtpof=true&sd=true
Demo	Code Demo - https://drive.google.com/file/d/1ZSmHgL5F41oHvS4EOGp4xQwimXgLU46i/view?usp=sharing PPT Demo - https://drive.google.com/file/d/1Aoqt2eSBPWlHB3E_Q6Xm-qJNugcc3haA/view?usp=sharing

Note: *All diagrams and tables in this report are original from Team 1.*